



Chapter 10

The Linear Algebra of Graphs

“If people do not believe that mathematics is simple, it is only because they do not realize how complicated life is.” – John von Neumann

10.1 Introduction

Graphs are encountered in many real-world settings, such as the Web, social networks, and communication networks. Furthermore, many machine learning applications are conceptually represented as optimization problems on graphs. Graph matrices have a number of useful algebraic properties, which can be leveraged in machine learning. There are close connections between kernels and the linear algebra of graphs; a classical application that naturally belongs to both fields is *spectral clustering* (cf. Section 10.5).

This chapter is organized as follows. The next section introduces the basics of graphs and representations with adjacency matrices. The structural properties of the powers of adjacency matrices are discussed in Section 10.3. The eigenvectors and eigenvalues of graph matrices are discussed in Section 10.4. The linear algebra of graph clustering is explored in Section 10.5, whereas the linear algebra of graph ranking algorithms is explored in Section 10.6. The linear algebra of graphs with poor connectivity properties is discussed in Section 10.7. Machine learning applications of graphs are discussed in Section 10.8. A summary is given in Section 10.9.

10.2 Graph Basics and Adjacency Matrices

A graph, which is sometimes also referred to as a *network*, is a structure that is used to represent “relationships” among objects. The objects may be of any type, such as Web pages, social network actors, or chemical elements. Similarly, the relationships may be of a (corresponding) application-relevant type, such as Web links, social network friendships, or chemical bonds. For example, the chemical compound *Acetaminophen* and its associated graph structure are illustrated in Figure 10.1(a) and (b), respectively. A graph representing a social network is illustrated in Figure 10.1(c).

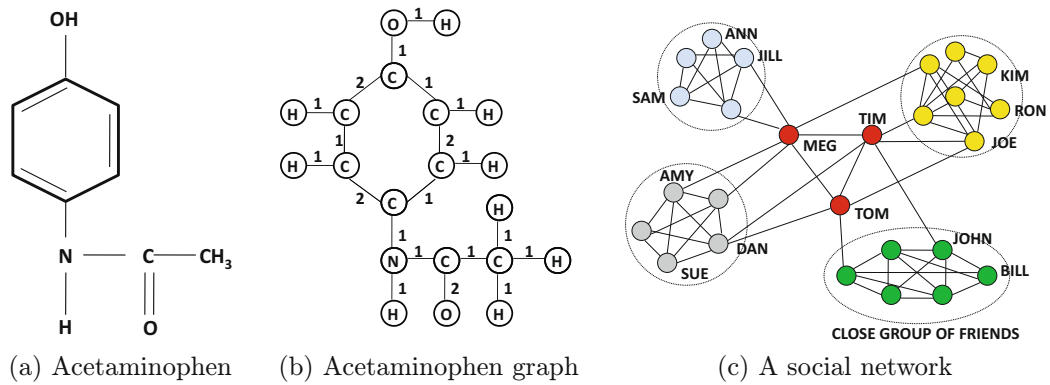


Figure 10.1: Examples of undirected graphs

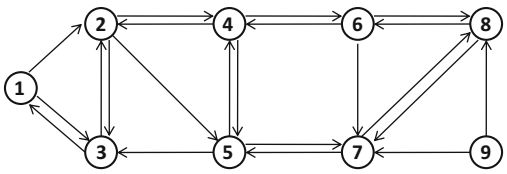


Figure 10.2: A directed graph

The objects in a graph are referred to as *vertices*, and the relationships among them are referred to as *edges*. A vertex is also sometimes referred to as a *node*. Throughout this book, we use the term “vertex” and “node” interchangeably. A graph G is denoted by the pair (V, E) , where V is a set of vertices (nodes), and E is a set of edges. If the graph contains n vertices, it is assumed that the vertices are $V = \{1 \dots n\}$. Similarly, each edge $(i, j) \in E$ represents a connection between the vertices i and j .

Graphs may be *directed* or *undirected*. In directed graphs, each edge has a direction. For example, Web links have a direction from the source page to the destination page. The source of an edge is referred to as its *tail* and the destination is referred to as its *head*. Therefore, edges are shown using arrows, where the head corresponds to the end containing an arrowhead. An example of a directed graph is illustrated in Figure 10.2. On the other hand, edges do not have direction in undirected graphs. For example, a Facebook friendship link or a chemical bond does not have direction. All the graphs illustrated in Figure 10.1 are undirected graphs. An undirected graph may be converted into a directed graph by replacing each undirected edge with a pair of directed edges in opposite directions.

Finally, graphs may be *unweighted* or *weighted*. In an unweighted graph, an edge may be present or absent between two vertices, and there is no “strength” associated with a specific edge. In algebraic terms, the representation is binary and the relationship between a pair of vertices has a value of either 1 or 0, depending on whether or not an edge is present between the pair. On the other hand, in many applications, the relationship might have a weight associated with it. For example, a chemical bond has a strength corresponding to the number of shared electrons. Correspondingly, weights are shown in Figure 10.1(b). In an email network, the weight of an edge from one participant to another might correspond to the number of messages sent along that edge. Since weighted graphs are more general, the graphs discussed in this chapter are always associated with nonnegative weights.

In an undirected graph, the *degree* of a vertex is defined as the number of incident edges at that vertex. For example, in Figure 10.1(c), the degree of the vertex corresponding to Sam is 4. Since every edge is incident on two vertices, the sum of the degrees of the vertices in an undirected graph with m edges is always equal to $2m$. In the case of a directed graph, it makes sense to talk about the *indegree* and the *outdegree* of a vertex. The indegree of a vertex is the number of incoming edges at a vertex, whereas the outdegree is the number of outgoing edges. For example, the indegree of vertex 1 in Figure 10.2 is 1, whereas its outdegree is 2. The sum of the indegrees over all vertices and the sum of the outdegrees over all vertices are both equal to the number of edges m . This is because each edge is incident on exactly one vertex as an incoming edge, and it is incident on one vertex as an outgoing edge. All definitions of vertex degrees can be generalized to the weighted case by adding the weights of the edges instead of using a default weight of 1 for each edge.

Basic Structures in Graphs

A *walk* is any sequence $i_1, i_2, \dots, i_k$ of vertices, so that an edge exists from each i_r to i_{r+1} . In the case of directed graphs, the tail of the edge must be at i_r and the head must be at i_{r+1} . In undirected graphs, an edge can be traversed in both directions. There is no restriction on repetition of vertices within a walk. In Figure 10.2, the sequence 2, 3, 1, 2, 4 of vertices is a walk. A *path* is any sequence of vertices $i_1, i_2, \dots, i_k$ of vertices, so that an edge exists from each i_r to i_{r+1} and there is no repetition of vertices. In directed graphs, the direction of the edges must be from i_r to i_{r+1} . Therefore, every path is a walk, but not vice versa. In Figure 10.2, the sequence 3, 1, 2, 4 is a path. A *cycle* is any sequence of vertices $i_1 i_2 \dots i_k$, so that an edge exists between each successive pair of vertices, $i_1 = i_k$, and there is no other repeating vertex. In other words, a cycle is a closed and directed “loop” of vertices in a directed graph. A cycle is also a special case of a walk. For undirected graphs, a cycle is simply a closed loop of undirected edges. In a directed graph, the direction of all edges in a cycle must be the same. Directed graphs that do not contain cycles are referred to as *directed acyclic graphs*. For example, consider a directed graph containing three vertices $\{1, 2, 3\}$, and the directed edges $(1, 2)$, $(1, 3)$, and $(2, 3)$. This graph does not contain any directed cycle and is therefore a directed acyclic graph.

A *subgraph* of a graph is any subset of vertices and edges in the graph. Note that if an edge is included in the subgraph, its end points must be included as well. The subgraph of a graph $G = (V, E)$ induced by a set of vertices $V' \subseteq V$ is the graph $G' = (V', E')$, in which $E' \subseteq E$ contains all edges between vertices in V' .

Connectivity and Diameter

An undirected graph is referred to as *connected*, if a path exists between each pair of vertices. An undirected graph that is not connected can be divided into a number of *connected components*. A connected component is a subset of vertices from the original graph, so that the subgraph induced by that vertex set is connected. Examples of both connected and unconnected undirected graphs are shown in Figure 10.3(a) and (b), respectively. The graph in Figure 10.3(b) has two connected components.

A directed graph is referred to as *strongly connected*, if a directed path exists between each pair of vertices in either direction. In other words, for any given pair of vertices $[i, j]$, a path must exist from i to j , and a path must also exist from j to i . For example, a graph corresponding to a single cycle of vertices is strongly connected. On the other hand, a single directed path of vertices or a directed acyclic graph is not strongly connected.

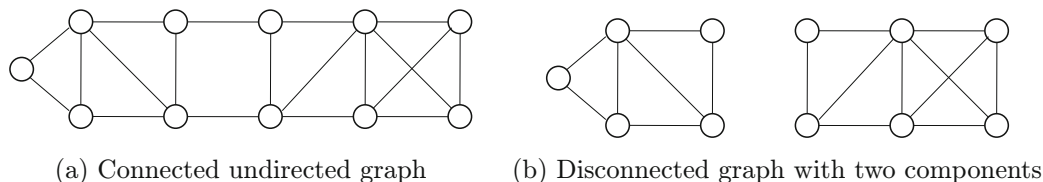


Figure 10.3: Examples of connected and disconnected undirected graphs

(because *directed* paths do not exist between specific *ordered* pairs of vertices). The graph in Figure 10.2 is not strongly connected, because a directed path does not exist from vertex 7 to vertex 9. As we will see later, strongly connected graphs have useful algebraic properties.

The *distance* or *shortest path* between a pair of vertices in a directed graph is defined as the least number of edges on a directed path between them. The *diameter* of a directed graph is defined as the largest distance between two vertices in the graph. Note that the distance from vertex i to vertex j might be different from that from vertex j to vertex i . Therefore, one needs to compute the distances between all $n(n-1)$ ordered pairs of vertices in the graph and compute the largest among them in order to compute the graph diameter. If no directed path exists between a particular pair of vertices, then the diameter of the graph is ∞ . Therefore, a directed graph needs to be strongly connected in order for its diameter to be finite. For example, the diameter of the directed graph in Figure 10.2 is ∞ because no directed path exists from vertex 7 to vertex 9.

In *undirected* graphs, the shortest path distance from vertex i to j is the same as that from vertex j to i . If no path exists between a pair of vertices, it means that the graph is *disconnected*, and the distance between this vertex pair is ∞ . The diameter of an undirected graph is the maximum of the shortest path distances between each pair of vertices. The diameter of a disconnected graph [like Figure 10.3(b)] is ∞ .

Graph Adjacency Matrix

The adjacency matrix of an undirected graph is a special case of that of a directed graph, because each undirected edge can be replaced with two directed edges in opposite directions of equal weight. Therefore, we will first discuss the more general case of directed graphs.

For a directed graph containing n vertices and m edges, a square $n \times n$ matrix $A = [a_{ij}]$ is defined in which the value of a_{ij} is the weight of the edge from vertex i to vertex j . If no edge exists from vertex i to vertex j , the value of that entry is 0. Therefore, the adjacency matrix of a directed graph with m edges will contain m non-zero entries. In the case of unweighted graphs, all entries in the matrix are 0s or 1s. It is common for the diagonal entries of an adjacency matrix to be 0s, because self-loops are extremely uncommon in graphs. The adjacency matrix of a directed graph is usually asymmetric because a_{ij} is typically not the same as a_{ji} . On the other hand, an undirected graph with m edges will have $2m$ non-zero entries, and it will be symmetric because a_{ij} has the same value as a_{ji} . The symmetric nature of undirected graph adjacency matrices simplifies their linear algebra, because they have real-valued and orthonormal eigenvectors.

Normalized Adjacency Matrices

There are several ways in which graph adjacency matrices are normalized. The goal of normalization is to prevent a few vertices with many incident edges from dominating the

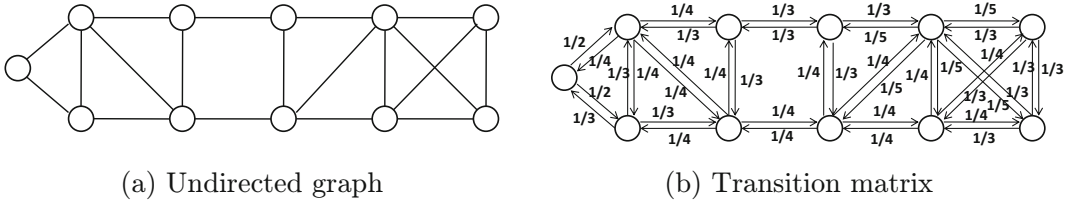


Figure 10.4: An undirected graph and its random walk graph. Note that asymmetric normalization makes a symmetric adjacency matrix asymmetric

algebraic properties of the graph. Most real-world graphs have *power-law degree distributions* [43], as a result of which the sum of the degrees of a tiny fraction of the vertices often form the vast majority of the sum of the degrees of all vertices in the full graph. As a result, the structure of the edges incident on these vertices dominate any type of analysis or the results of a machine learning algorithm applied to the entire network. This is undesirable because the structural behavior of high-degree nodes is often caused by spam and other irrelevant/noisy edges.

Some forms of normalization have a probabilistic interpretation, which are useful in real applications. The first type of normalization is *asymmetric* normalization, in which every row is normalized to sum to 1 unit. Therefore, we sum the elements of each row, and divide each element in that row by this sum. The result of this type of normalization is to create a *stochastic transition matrix*, that converts the adjacency matrix into the transition matrix of a *Markov chain*. This transition matrix defines a random walk at the graph, where the outgoing probabilities at each vertex define the probability of traversing along that edge. The resulting graph is referred to as a *random walk graph*. Note that this type of normalization results in asymmetric weights even for an undirected graph (for which the unnormalized adjacency matrix is symmetric). An example of asymmetric normalization is shown in Figure 10.4, where the original graph with binary edge weights is shown on the left, whereas the normalized graph (i.e., random walk graph) is shown on the right. It is noteworthy that this type of asymmetric normalization can also be applied to a directed graph. In such a case, the weight of each edge is divided by the sum of the weights of the *outgoing* edges at a vertex. The goal is again to interpret each edge weight as a random walk probability *out* of a given vertex.

Symmetric normalization is generally defined for undirected graphs. Therefore, one starts with a symmetric adjacency matrix and the goal is to preserve its symmetry in the normalization process. In symmetric normalization, we sum up the nonnegative entries of the i th row to create the sum δ_i . Since the matrix is symmetric, the sum of the elements of the i th column is also δ_i . In other words, we have the following:

$$\delta_i = \sum_{j=1}^n a_{ij} = \sum_{j=1}^n a_{ji}$$

In symmetric normalization, we divide each entry with the *geometric mean* of its row and column sums. The resulting similarity value s_{ij} is defined as follows:

$$s_{ij} \Leftarrow \frac{a_{ij}}{\sqrt{\delta_i \delta_j}}$$

Note that in asymmetric normalization, we always use $p_{ij} \Leftarrow a_{ij}/\delta_i$, and therefore the sum of each row is 1. Here, p_{ij} represents the probability of transition to vertex j from vertex i in the random walk graph.

One can also represent the above normalizations algebraically in the form of matrix multiplication. Let $A = [a_{ij}]$ be the original $n \times n$ (undirected) adjacency matrix, and Δ be a diagonal $n \times n$ matrix in which the i th diagonal entry is $\delta_i = \sum_j a_{ij}$. The matrix Δ is referred to as the *degree matrix* of A . It is noteworthy that the degree matrix incorporates information about the weights a_{ij} of edges; the values on the diagonal of Δ will be the aggregate *weights* of incident edges rather than the *number* of incident edges. We occasionally refer to the matrix Δ as the *weighted* degree matrix, although referring to it simply as “degree matrix” is more common. Let $P = [p_{ij}]$ be the asymmetrically normalized stochastic transition matrix, and $S = [s_{ij}]$ be the symmetrically normalized adjacency matrix. Then, the asymmetrically and symmetrically normalized matrices are defined as follows:

$$\begin{aligned} P &= \Delta^{-1}A && \text{[Asymmetric Normalization]} \\ S &= \Delta^{-1/2}A\Delta^{-1/2} && \text{[Symmetric Normalization]} \end{aligned}$$

As we will see, these two related matrices play an important role in many network applications including clustering, classification, and *PageRank* computation.

10.3 Powers of Adjacency Matrices

Consider a binary adjacency matrix A of a directed graph in which each entry is either 0 or 1. The powers of A are related to the number of walks of specific lengths:

Property 10.3.1 *Let A be a binary adjacency matrix of a graph. The value of the (i, j) th entry in A^k is equal to the number of walks from i to j of length exactly k .*

This result can be shown using induction. It is easy to see that $A^1 = A$ contains all walks of length exactly 1. Now, if the (i, j) th entry of A^k contains the number of walks of length k from i to j , then the number of walks of length $(k + 1)$ can be obtained by summing up the number of walks of length k from i to all vertices connected to j (i.e., all vertices with a value of 1 in the j th column of A). This value is precisely given by the (i, j) th entry of $A^k A = A^{k+1}$. What happens when the edges are weighted? In such a case, the value of A^k is equal to the number of weighted walks, where the contribution of each walk is equal to the product of the values of a_{ij} on that walk.

The number of walks of length *at most* t from i to j is contained in the matrix $W(t) = \sum_{k=0}^t A^k$. Note that A^0 is the identity matrix, and we are including it in the summation because every vertex is reachable from itself with a walk of length 0. In many cases, the zeroth order term is omitted because walks are assumed to have non-zero length. In general, this summation does not converge as t increases. For example, if A corresponds to a cycle of two vertices, it is easy to show the following:

$$\begin{aligned} A^r &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} && \text{[Odd } r] \\ A^r &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} && \text{[Even } r] \end{aligned}$$

This is because there is only one way of walking from a vertex to itself in a specified number of even steps, and only one way of walking across the two vertices in a specified number of

odd steps. An infinite summation over these matrices will not yield a converging summation. Furthermore, the entries of A^r could themselves blow up over an infinite summation (in some types of matrices).

However, by allowing for a decay factor $\gamma < 1$, it is possible for this summation to converge. From a semantic perspective, this means that a walk of length r is weighted by γ^r . Even though there might be more walks of greater length, the decay factor ensures that the infinite summation eventually converges. Clearly, the choice of γ required for convergence depends on the structural properties of the graph; interestingly, these structural properties can be captured by the eigendecomposition of the underlying adjacency matrix. Choosing γ less than the reciprocal of the largest absolute eigenvalue of A ensures that the powers of $(\gamma A)^k$ do converge over an infinite summation. In other words, we are causing a decay at the multiplicative factor of $\gamma < 1$, and then summing up the weights of all walks between a pair of vertices.

This result follows from the fact that if λ is an eigenvalue of A , then $\gamma\lambda$ is an eigenvalue of the matrix $A_\gamma = \gamma A$. This is because $\det(A_\gamma - \lambda\gamma I) = \gamma^n \det(A - \lambda I)$. As a result, by choosing γ to be less than the reciprocal of the largest absolute magnitude of the eigenvalues in A , the largest absolute magnitude of the eigenvalues of the matrix $A_\gamma = \gamma A$ is strictly less than 1. The following result can be easily shown:

Lemma 10.3.1 *Let A be a matrix for which all eigenvalues have absolute magnitude less than $1/\gamma$ for $\gamma > 0$. Then, the following can be shown:*

$$\lim_{r \rightarrow \infty} (\gamma A)^r = 0$$

Proof Sketch: We denote γA with the matrix A_γ , and all its eigenvalues have absolute magnitude less than 1. Any matrix can be converted into Jordan normal form (cf. Section 3.3.3) with possibly complex eigenvalues as follows:

$$A_\gamma = VJV^{-1}$$

Here, J is an upper-triangular matrix in Jordan normal form in which the diagonal entries contain the (possibly complex) eigenvalues, each with magnitude less than 1. In such a case, we can show the following:

$$A_\gamma^r = VJ^rV^{-1}$$

As r goes to ∞ , the matrix J^r can be shown¹ to go to 0. Therefore, the matrix A_γ^r converges to the zero matrix as well. ■

This result provides an approach for computing the decay-weighted sum of walks between any pair of vertices.

Lemma 10.3.2 *Given a directed graph with adjacency matrix A in which the largest eigenvalue is less than $1/\gamma$, the weighted sum of all decayed walks between each pair of vertices is contained in the matrix $(I - \gamma A)^{-1} - I$.*

$$\sum_{r=1}^{\infty} (\gamma A)^r = (I - \gamma A)^{-1} - I$$

¹The diagonal entries in the upper-triangular matrix J^r are powers of those in J , which will go to 0. Such a strictly triangular matrix is known to be nilpotent, and, therefore, J^r will eventually go to 0.

Proof Sketch: All eigenvalues of γA have magnitude less than 1. Each eigenvalue of γA has a corresponding eigenvalue in $(I - \gamma A)$, and the two eigenvalues sum to 1 with the same eigenvector. Therefore all eigenvalues of $(I - \gamma A)$ are non-zero, and the matrix is non-singular. In other words, we can multiply both sides of the above equation with $(I - \gamma A)$ without affecting the correctness of the above result. Multiplying each side with $(I - \gamma A)$ yields the matrix γA for both sides. This proves the result. ■

The matrix $(I - \gamma A)^{-1}$ is very useful because, the (i, j) th entry tells us about the level of indirect connectivity from vertex i to j even when this pair of vertices is not directly connected. This matrix contains all the *Katz measures* between pairs of vertices. The Katz measure is generally used for undirected graphs, although it can also be applied to directed graphs.

Definition 10.3.1 (Katz Measure) *Given the adjacency matrix A of an undirected graph, the Katz measure between vertices i and j is the (i, j) th entry of $(I - \gamma A)^{-1} - I$. Here, γ is a decay parameter used for computing the Katz measure.*

The Katz measure can be used for *link prediction*. In this problem, the goal is to discover pairs of vertices between which links are likely to form in the future in a graph of interest (e.g., social network). Clearly, if many (short) walks exist between a pair of vertices, links are more likely to form between them in the future. For example, one is more likely to form links with the friends of one's friends in a social networks.

This problem of link prediction can be viewed in a similar manner to that of the problem of recommendations with implicit feedback. To solve this problem, we first set a small subset of the non-zero entries in A to 0 to obtain A' , and save those edges as a *validation set* A_v . We also add some of the zero edges in A to A_v . One can compute the inverse of $(I - \gamma A')$ at different values of γ and select the one at which the link predictions on the validation set A_v are the most accurate. Once the value of γ has been determined, we compute the matrix $(I - \gamma A)^{-1}$ (with all edges included) in order to rank the edges in the order of the likelihood of forming links.

The powers of the adjacency matrix can also be used to characterize the diameter and connectivity of both undirected and directed graphs. Note that if the diameter of a graph is d , then a path of length at most d exists between each pair of vertices. In other words, if $r_{ij} \leq d$ be the length of the shortest path between an arbitrary pair of vertices i and j , then the (i, j) th entry of $A^{r_{ij}}$ will be non-zero. This implies the following way of defining the diameter of a graph in terms of the powers of the adjacency matrix.

Property 10.3.2 *The diameter of a (directed or undirected) graph with adjacency matrix A is the smallest value of d for which all entries of the matrix $\sum_{k=0}^d A^k$ are non-zero. For any connected, undirected graph or strongly connected, directed graph, the value of d is at most $n - 1$.*

When an undirected graph is not connected or a directed graph is not strongly connected, the diameter of the graph is ∞ . Since all graphs with finite diameter have a diameter value at most $n - 1$, this provides a simple approach for testing the connectivity of a (directed or undirected) graph with the use of powers of the adjacency matrix.

Lemma 10.3.3 (Connectivity of Undirected Graph) *An undirected graph with $n \times n$ adjacency matrix A is connected if and only if every entry of the matrix $\sum_{k=0}^{n-1} A^k$ is non-zero.*

Lemma 10.3.4 (Strong Connectivity of Directed Graph) *A directed graph with $n \times n$ adjacency matrix A is strongly connected if and only if every entry of the matrix $\sum_{k=0}^{n-1} A^k$ is non-zero.*

A simple approach for testing connectivity is to compute the matrix containing all Katz measures and checking if all nondiagonal entries are non-zero.

Undirected and directed graphs that are not connected/strongly connected will have missing edges between specific sets of vertices in the matrix $\sum_{k=0}^{n-1} A^k$. The missing edges in $\sum_{k=0}^{n-1} A^k$ will result in a specific type of block structure of the non-zero entries. For example, consider a directed graph in which the vertices can be divided into sets 1 and 2. Assume that the vertices in each of the sets 1 and 2 are strongly connected. Furthermore, edges exist from vertices in set 1 to vertices in set 2, but no edges exist in the other direction. This type of graph is not strongly connected and its adjacency matrix can always be converted into a *block upper-triangular form* by appropriately reordering the vertices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix}$$

Note that the blocks along the diagonal are always square, but the blocks above the diagonal might not be square. It is not very difficult to verify that no matter how many times we exponentiate the matrix A , the lower block of zeros will stay as zeros. Similarly, a disconnected, undirected adjacency matrix can be converted into *block diagonal form* like the one below:

$$A = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix}$$

It is relatively easy to verify that exponentiating this matrix any number of times will only result in the individual blocks being exponentiated:

$$A^k = \begin{bmatrix} A_{11}^k & 0 \\ 0 & A_{22}^k \end{bmatrix}$$

The two blocks A_{11} and A_{22} do not interact with one another in matrix multiplication.

10.4 The Perron-Frobenius Theorem

In the following, we will show some key properties of the eigenvectors of adjacency matrices in both directed and undirected graphs. The Perron-Frobenius theorem applies to the general case of *directed* graphs, with some additional simplifications for undirected graphs. All graph adjacency matrices (directed or undirected) have nonnegative entries, as a result of which the underlying eigenvectors have special properties. Undirected graphs have more interesting properties than directed graphs, because the symmetry in their adjacency matrices assures orthonormal eigenvectors and real eigenvalues. The adjacency matrices of directed graphs may contain both real and complex eigenvalues. This family of results is generally referred to as the *Perron-Frobenius theorem*.

Property 10.4.1 *The adjacency matrices of directed graphs might have one or more complex eigenvalues. Furthermore, a directed graph is not even guaranteed to have a diagonalizable adjacency matrix even after allowing for complex eigenvalues.*

As an example of non-diagonalizability, consider a directed graph of four nodes, which has bidirectional edges between each *consecutive* node pair $[i, i + 1]$ and a single unidirectional edge from node 1 to node 4:

$$N = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Without the unidirectional edge, the graph is symmetric and diagonalizable with real and orthonormal eigenvectors. However, the addition of a single edge from node 1 to node 4 makes the adjacency matrix non-diagonalizable. On computing $\det(N - \lambda I)$, it can be shown that this matrix has the characteristic polynomial $\lambda^4 - 3\lambda^2$, which corresponds to the eigenvalues $\{-\sqrt{3}, \sqrt{3}, 0, 0\}$. Therefore, the eigenvalue 0 is repeated. However, there is only one eigenvector $[0, 1, 0, -1]^T$ with eigenvalue 0, since the matrix has rank 3. Therefore, this adjacency matrix is not diagonalizable.

As an example of a directed graph with a diagonalizable adjacency matrix and complex eigenvalues, consider the following adjacency matrix, which contains a directed cycle of three vertices:

$$A = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad (10.1)$$

Note that this matrix does have one real-valued eigenvector $[1, 1, 1]^T$ with an eigenvalue of 1. The other two eigenvalues can be shown to be $(-1 + i\sqrt{3})/2$ and $(-1 - i\sqrt{3})/2$, which are obviously complex. The corresponding eigenvectors are also complex. All eigenvalues can be shown to be the real and complex cube roots of unity because the characteristic polynomial is $\lambda^3 - 1$. It is noteworthy that we did get at least one real eigenvector-eigenvalue pair. Furthermore, this eigenvector is the largest eigenvector in absolute² magnitude, although the other two eigenvectors also have a magnitude of 1. This is not a coincidence. It can be shown that the adjacency matrix of any strongly connected directed graph will have at least one real eigenvector-eigenvalue pair, which is also the dominant pair. The adjacency matrix of a strongly connected graph is said to be *irreducible*.

Definition 10.4.1 (Irreducible Matrix) *The adjacency matrix of a directed graph is said to be irreducible, if and only if the underlying graph is strongly connected.*

An adjacency matrix that is not irreducible is said to be *reducible*. Note that if the graph is not strongly connected (i.e., its adjacency matrix is reducible), then either the graph is completely disconnected, or its vertices can be partitioned into two sets (i.e., a *cut* can be created) such that edges between the two sets point in only one direction. In other words, the matrix can be expressed in the following *block upper-triangular form*:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix} \quad (10.2)$$

In terms of walks on the directed graph, it means that once one moves from block 1 to block 2 via edges in A_{12} , it is impossible to come back to any vertex in block 1. In the above matrix it is assumed that the vertices are ordered so that all vertices in one component occur before all vertices in another component.

²The magnitude of the complex number $a + ib$ is $\sqrt{a^2 + b^2}$.

The Perron-Frobenius theorem applies only to strongly connected graphs, which are represented by irreducible adjacency matrices. The primary focus of this result is on the eigenvector of largest *magnitude*, which is also referred to as the *principal eigenvector*. Note that the left eigenvectors and right eigenvectors of an asymmetric matrix (like a directed graph) are different. The general version of the Perron-Frobenius theorem is stated as follows:

Theorem 10.4.1 (Perron-Frobenius Theorem for Directed Graphs) *Let A be a square, irreducible adjacency matrix with nonnegative entries for a directed graph containing n vertices. Then, one of the largest eigenvalues of A (in absolute magnitude) is always real-valued and positive (denoted by λ_{max}), and the multiplicity of this (positive) eigenvalue is 1. However, other complex or negative eigenvalues could exist with absolute magnitude λ_{max} . The following results also hold true:*

- The unique left eigenvector and unique right eigenvector corresponding to the real and positive eigenvalue λ_{max} contains only real and strictly positive entries.
- The real and positive eigenvalue λ_{max} satisfies the following:

$$\begin{aligned} \text{average}_i \sum_j a_{ij} &\leq \lambda_{max} \leq \max_i \sum_j a_{ij} \\ \text{average}_j \sum_i a_{ij} &\leq \lambda_{max} \leq \max_j \sum_i a_{ij} \end{aligned}$$

This means that for unweighted matrices, the largest eigenvalue lies between the average and maximum indegree (and outdegree). Therefore, the minimum of the maximum indegree and maximum outdegree can be used to provide an upper bound on λ_{max} .

- **Special case of stochastic transition matrices:** *Since stochastic transition matrices have a weighted outdegree of 1 for each vertex, the largest eigenvalue is 1 according to the above results. The corresponding right eigenvector of a stochastic transition matrix P is the n -dimensional column vector of 1s because each row of P sums to 1. The corresponding left eigenvector, referred to as the **PageRank** vector, is the solution to $\bar{\pi}^T P = \bar{\pi}^T$.*

It is noteworthy that even though the largest eigenvalue in magnitude is positive, complex or negative eigenvalues might exist with the same absolute magnitude λ_{max} . As a specific example, consider the directed cycle of three vertices in the adjacency matrix of Equation 10.1. The eigenvalues of this matrix are the three real or complex cube-roots of 1, which are 1, $(-1 + i\sqrt{3})/2$ and $(-1 - i\sqrt{3})/2$. All three roots have an absolute magnitude of 1. In general, it can be shown that a directed cycle of n vertices has n eigenvalues corresponding to the n real and complex n th roots of 1. These values can be shown to be $\exp(2i\pi t/n) = \cos(2\pi t/n) + i\sin(2\pi t/n)$ for $t \in \{0 \dots n-1\}$. All real and complex eigenvalues have an absolute magnitude of 1. If the graph is reducible, there might be multiple eigenvectors corresponding to the largest (positive) eigenvalue. Furthermore, the principal eigenvector is no longer guaranteed to contain only positive entries in such a case.

A simplified version of the Perron-Frobenius theorem also applies to undirected graphs. In undirected graphs, the adjacency matrix is symmetric and therefore all eigenvectors are real. Furthermore, the graph is always strongly connected, when viewed as a directed network (with two directed edges replacing each undirected edge). Correspondingly, one can state the Perron-Frobenius theorem for undirected graphs as follows:

Corollary 10.4.1 (Perron-Frobenius Theorem for Undirected Graphs) *Let A be a nonnegative adjacency matrix for an undirected, connected graph containing n vertices. Then, the largest eigenvalue $\lambda_{\max}$ of A (in absolute magnitude) is always real-valued and positive. There is only one eigenvalue with value $\lambda_{\max}$, although it is possible for a negative eigenvalue of the same magnitude to exist. The following results are true:*

- *All eigenvectors and eigenvalues are real. If the graph does not have self-loops, the sum of the eigenvalues is equal to the trace of A , which is 0. Therefore, some eigenvalues will always be negative. The eigenvalue $\lambda_{\max}$ is always at least as large as the most negative eigenvalue (in absolute magnitude).*
- *There is a single eigenvector corresponding to the eigenvalue $\lambda_{\max}$. The eigenvector only has strictly positive entries (after multiplying with -1 if necessary). The uniqueness means that the solution to $A\pi = \lambda_{\max}\pi$ is unique to within scaling.*
- *The largest eigenvalue satisfies the following:*

$$\text{average}_i \sum_j a_{ij} \leq \lambda_{\max} \leq \max_i \sum_j a_{ij}$$

This means that for unweighted matrices, the largest eigenvalue lies between the average and maximum degree.

Finally, the stochastic transition matrices of undirected graphs are not symmetric; however, they inherit some of the properties of undirected (symmetric) adjacency matrices from which they are derived. For example, the stochastic transition matrices of undirected graphs continue to have real eigenvectors and real eigenvalues like their symmetric counterparts.

Corollary 10.4.2 (Stochastic Transition Matrices of Undirected Graphs) *Let $P = \Delta^{-1}A$ be the normalized transition matrix of an undirected, connected graph with $n \times n$ adjacency matrix A and degree matrix Δ . The following results are true:*

- *All eigenvectors and eigenvalues are real. The largest eigenvalue in absolute magnitude is always unique and has a value of 1, although it is also possible to have an eigenvalue with value -1 .*
- *The single right eigenvector with eigenvalue of 1 corresponds to an n -dimensional column of 1s. This vector is a valid right eigenvector because each row sums to 1 in the stochastic transition matrix.*
- *The left eigenvector solution to $\pi^T P = \pi^T$ is unique to within scaling. This vector is referred to as the **PageRank** vector and all components are strictly positive.*

Note that even though the eigenvector with largest eigenvalue of 1 is unique, other eigenvectors could exist with an eigenvalue of -1 . Therefore, the eigenvector in largest *magnitude* is not unique. As a specific example, consider the stochastic transition matrix of an undirected graph with two vertices and a single edge between them. Both the adjacency matrix A and the stochastic transition matrix P have the following form:

$$A = P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

This graph has the same adjacency matrix as a directed cycle of two vertices. It has eigenvalues of -1 and $+1$.

The eigenvectors of normalized adjacency matrices are used extensively in machine learning for applications like spectral clustering and ranking. The left eigenvectors are used for applications like ranking nodes, whereas the right eigenvectors are used for spectral clustering. This will be the focus of subsequent sections.

10.5 The Right Eigenvectors of Graph Matrices

In this section, we will discuss the linear algebra of right eigenvectors of *undirected* graphs, which are used for a specific type of graph clustering, referred to as *spectral clustering*. Throughout this section, our assumption is that we are working with an undirected graph with a symmetric adjacency matrix, which ensures real eigenvalues and eigenvectors.

Consider an undirected graph with adjacency matrix A . Before applying any form of graph clustering, a preprocessing step is applied to the adjacency matrix A . The goal of this preprocessing step is to break the low-weight links in the graph and set their weights to 0. Doing so tends to differentially remove the inter-cluster links, and makes the clustering much cleaner. There are several ways in which this can be done, both of which require the use of a threshold parameter to decide which links to remove.

1. All non-zero entries in the adjacency matrix A that are less than a threshold value of ϵ are set to 0. In other words, if a_{ij} is less than ϵ , it is set to 0.
2. For each vertex i , we set all those entries a_{ij} to 0 if j is not among the κ -nearest neighbors of i for some user-driven parameter κ . However, this change will make the matrix asymmetric. Therefore, we prefer the use of *mutual* κ -nearest neighbors over the use of κ -nearest neighbors. In other words, we set a_{ij} and a_{ji} to 0 if and only if i and j are both κ -nearest neighbors of each other.

These types of methods are simply heuristics. One can use almost any reasonable heuristic to break weak links, as long as the final adjacency matrix A is symmetric. Keeping a symmetric adjacency matrix is important for ensuring real-valued eigenvectors and eigenvalues in spectral clustering. *Throughout this section, we will assume that the matrix A refers to an adjacency matrix in which the weak links have been removed.* An adjacency matrix is simply a form of a similarity matrix (or *kernel*) although it is not positive semidefinite because the diagonal elements are zeros. It is noteworthy that one often converts multidimensional data sets to graphs by treating data points as objects and the weights of edges as similarity values based on the Gaussian kernel (cf. Chapter 9). In fact, if we use such a graph without breaking the weak links, the resulting clustering is very similar to that of kernel k -means (with only small detail-oriented differences associated with feature selection and normalization).

The removal of weak links makes each entry in the adjacency matrix dependent on the values of other entries. This type of similarity function is referred to as a *data-dependent* kernel. Unlike a pure Gaussian kernel (cf. Chapter 9), a data-dependent kernel cannot be computed purely as a function of its two arguments and depends on the remainder of the similarity matrix. In this section, we provide three equivalent views of spectral clustering, which correspond to the kernel view, the Laplacian view, and the matrix factorization view.

10.5.1 The Kernel View of Spectral Clustering

Before discussing spectral clustering, we first discuss the most obvious kernel method obtained by treating the symmetric adjacency matrix of an undirected graph as a similarity

matrix. This discussion sets the stage for understanding spectral methods as variants of this kernel method. We assume that the adjacency matrix (after removing the weak links) is denoted by $A = [a_{ij}]$. The $n \times n$ adjacency matrix A of an undirected graph is a symmetric similarity matrix, although it is not positive semidefinite because the sum of its eigenvalues (i.e., matrix trace) is 0. Nevertheless, one can always diagonalize the symmetric adjacency matrix as $A = Q\Lambda Q^T$ and simply use the top- k columns of Q as the engineered representation. Any clustering algorithm like k -means can be applied on the embedding (cf. Section 9.5.1.1). The lack of positive semidefiniteness of the similarity matrix (i.e., the adjacency matrix) might lead one to assume that this is not a kernel method. However, this is not quite correct; we could also condition the matrix A by adding the absolute value $\gamma > 0$ of the most negative eigenvalue to each diagonal entry. The matrix $A + \gamma I = Q(\Lambda + \gamma I)Q^T$ is a positive semidefinite matrix *with exactly the same eigenvectors*. The eigenvalues are not used. This is slightly different from kernel k -means (cf. Section 9.5.2.1), which implicitly scales the eigenvectors with the square-root of the eigenvalues via the kernel trick. Spectral clustering ignores the scaling effect of eigenvalues, and works with *whitened* representations (cf. Section 7.4.6 of Chapter 7). Furthermore, it only uses the top- k eigenvectors, thereby replacing soft eigenvalue weighting with discrete selection. These types of normalization and feature selection differences always occur in cases where one uses feature engineering (as in spectral methods) rather than the kernel trick.

The approach discussed above is a kernel method (related to spectral clustering), but it is not precisely spectral clustering. One problem with using adjacency matrices (as similarity matrices) directly is that the entries of the matrix are dominated by a few vertices. Most real-world graphs satisfy *power-law degree distributions* [43] in which a tiny fraction (typically less than 1%) of the vertices account for most of the edges in the graph. As a result, the embedding is dominated by the topological structure of a small fraction of vertices, which is undesirable. Spectral clustering solves this problem with *vertex degree normalization*.

In symmetric normalization, we compute the degree matrix Δ , which is an $n \times n$ diagonal matrix containing the degree $\delta_i = \sum_j a_{ij}$ on the i th diagonal entry. Each entry a_{ij} is divided by the geometric mean of δ_i and δ_j in order to reduce the influence of high-degree vertices. As discussed in Section 10.2, the symmetrically normalized similarity matrix S is defined as follows:

$$S = \Delta^{-1/2} A \Delta^{-1/2} \quad (10.3)$$

We can diagonalize this similarity matrix $S = Q_{(s)} \Lambda Q_{(s)}^T$, and then use the top- k columns of $Q_{(s)}$ (i.e., largest eigenvectors) as the $n \times k$ matrix $Q_{(s),k}$ containing the embedding. We subscript the matrix $Q_{(s),k}$ containing the embedding with ‘(s)’ to emphasize symmetric normalization. The i th row of $Q_{(s),k}$ contains the k -dimensional embedding of the i th vertex. This embedding is referred to as the *Ng-Jordan-Weiss* embedding [98]. Any clustering algorithm like k -means can be applied to this embedding. It is also common to normalize each *row* of the $n \times k$ matrix $Q_{(s),k}$ to unit norm just before applying the k -means algorithm. Note that normalizing each row of $Q_{(s),k}$ will result in an embedding matrix in which the columns are no longer normalized.

A related variation of this approach is the *Shi-Malik algorithm* [115], which uses the stochastic transition matrix instead of symmetrically normalized matrix. In other words, the normalization is asymmetric:

$$P = \Delta^{-1} A$$

One can also view P as a similarity matrix; however, the main problem is that it is asymmetric; therefore, it does not even make sense to talk about positive semidefiniteness.

As discussed in Section 9.3.3 of Chapter 9, it is still possible to extract embeddings from asymmetric similarity matrices if one is willing to relax the definition of similarity to allow asymmetry. In this case, asymmetric decomposition is used:

$$P = Q_{(a)} \Lambda Q_{(a)}^{-1}$$

Note that this decomposition always contains real-valued eigenvectors and eigenvalues according to the Perron-Frobenius theorem for stochastic transition matrices (cf. Corollary 10.4.2). The columns of $Q_{(a)}$ contain the right eigenvectors of P . We subscript the matrix with ‘(a)’ to emphasize the fact that it is extracted from an asymmetric similarity matrix. The top- k columns of $Q_{(a)}$ are extracted in order to create an $n \times k$ embedding matrix $Q_{(a),k}$. The i th row of this matrix contains the k -dimensional embedding of the i th vertex, and it is referred to as the Shi-Malik embedding. Any off-the-shelf clustering algorithm can be applied to this embedding. We make an observation about the top eigenvector in this embedding, which is also stated in the Perron-Frobenius result for stochastic matrices (cf. Corollary 10.4.2):

Property 10.5.1 *The largest eigenvector in the Shi-Malik embedding is a column of 1s.*

This particular eigenvector is not very informative from a clustering point of view and is sometimes discarded (although including it does not seem to make much of a difference).

10.5.1.1 Relating Shi-Malik and Ng-Jordan-Weiss Embeddings

The Shi-Malik and Ng-Jordan-Weiss embeddings are almost identical—in fact, one can obtain either embedding from the other with a simple post-processing step. This is because the symmetrically normalized matrix S and transition matrix P are related as $S = \Delta^{1/2} P \Delta^{-1/2}$, which makes the matrices similar. Therefore, their eigenvalues are the same and the eigenvectors are related.

Lemma 10.5.1 *Let Δ be the degree matrix of adjacency matrix A . Let S and P be the symmetrically normalized and the stochastic transition matrices of A . Then, $\bar{x}$ is an eigenvector of P with eigenvalue λ , if and only if $\sqrt{\Delta}\bar{x}$ is an eigenvector of S with eigenvalue λ .*

Proof: We show that both of the above statements are true if and only if $\bar{x}$ is a generalized eigenvector of A satisfying $A\bar{x} = \lambda\Delta\bar{x}$.

First, we note that $P\bar{x} = \lambda\bar{x}$ is true if and only if $\Delta^{-1}A\bar{x} = \lambda\bar{x}$, which is the same as saying that $A\bar{x} = \lambda\Delta\bar{x}$.

Second, we note that $S[\sqrt{\Delta}\bar{x}] = \lambda[\sqrt{\Delta}\bar{x}]$ is true if and only if $\Delta^{-1/2}A\bar{x} = \lambda[\sqrt{\Delta}\bar{x}]$, which is the same as saying that $A\bar{x} = \lambda\Delta\bar{x}$. This completes the proof. ■

Since the first eigenvector of the Shi-Malik embedding is a column of 1s, it follows that the first eigenvector of the Ng-Jordan-Weiss embedding is proportional to $\sqrt{\Delta}[1, 1, \dots, 1]^T$, which is the same as $[\sqrt{\delta_1}, \sqrt{\delta_2}, \dots, \sqrt{\delta_n}]^T$.

Corollary 10.5.1 *The first eigenvector of the symmetrically normalized adjacency matrix $S = \Delta^{-1/2}A\Delta^{-1/2}$ is proportional to an n -dimensional vector containing the square-roots of the weighted vertex degrees.*

An important point is that the Ng-Jordan-Weiss embedding normalizes the rows before applying k -means, whereas the Shi-Malik approach does not normalize the rows before applying k -means. In other words, the former is *row-normalized* before applying k -means, whereas the latter is *column-normalized* before applying k -means. This implies the following unified extraction of both embeddings using the symmetric kernel matrix $S = \Delta^{-1/2}A\Delta^{-1/2}$:

Lemma 10.5.2 *Let R be the $n \times k$ matrix containing the top- k (unit normalized) eigenvectors of the symmetric matrix $S = \Delta^{-1/2}A\Delta^{-1/2}$ in its columns. Then, both the Shi-Malik and Ng-Jordan-Weiss embeddings can be obtained from R using the following postprocessing steps:*

- *The Ng-Jordan-Weiss embedding is obtained by normalizing each row of the $n \times k$ matrix R to unit norm.*
- *The Shi-Malik embedding is obtained by normalizing each column of the $n \times k$ matrix $\Delta^{-1/2}R$ to unit norm.*

This observation shows that both methods are very similar, and differ only in terms of the minor post-processing steps of scaling/normalizing the rows/columns of the same matrix.

10.5.2 The Laplacian View of Spectral Clustering

The Laplacian view of spectral clustering is the most popular way of presenting it in most textbooks and survey papers [84]. We have not chosen this presentation of spectral clustering as the primary one because the relationship of spectral clustering to kernel methods is more fundamental. Nevertheless, we will briefly present the Laplacian view and discuss its relationship to the kernel view. An additional contribution of the Laplacian view (beyond the kernel view) is an unnormalized variant of the algorithm (although it is rarely used).

10.5.2.1 Graph Laplacian

Let A be the adjacency matrix of an undirected graph, and Δ be its degree matrix. The notations P and S correspond to the stochastic transition matrix and the symmetrically normalized adjacency matrix, respectively. The Laplacian is almost always used in undirected graph applications, although one can easily extend the definitions below to directed graphs. There are three types of Laplacians that are commonly used in graph applications. The first is the *unnormalized* Laplacian L :

$$L = \Delta - A \quad (10.4)$$

The Laplacian of an undirected graph is symmetric because both Δ and A are symmetric. The Laplacian is always a singular matrix because the sum of each row is 0. Therefore, one of the eigenvalues of the Laplacian is 0.

The asymmetrically normalized Laplacian L_a applies a one-sided normalization to the matrix L introduced above:

$$L_a = \Delta^{-1}L = \Delta^{-1}(\Delta - A) = I - P \quad (10.5)$$

Here, P is the stochastic transition matrix $P = \Delta^{-1}A$, which was introduced slightly earlier. Therefore, the asymmetrically normalized Laplacian is closely connected to the stochastic transition matrix P . This Laplacian is not symmetric, because P is not symmetric. The symmetrically normalized Laplacian L_s is defined in a similar way, except that two-sided normalization is applied to L :

$$L_s = \Delta^{-1/2}L\Delta^{-1/2} = \Delta^{-1/2}(\Delta - A)\Delta^{-1/2} = I - S \quad (10.6)$$

Therefore, the symmetrically normalized Laplacian is closely related to the symmetric similarity matrix S .

The graph Laplacian has some interesting interpretations in terms of embedding vertices in multidimensional space. In the simplest case, consider a setting in which each vertex i is embedded to the real number x_i . Therefore, one can create a normalized vector $\bar{x} = [x_1 \dots x_n]^T$, so that $\bar{x}^T \bar{x} = 1$. First, let us examine the unnormalized Laplacian L of Equation 10.4. One can show the following result:

Lemma 10.5.3 *If L is the unnormalized Laplacian of an undirected adjacency matrix, then $\bar{x}^T L \bar{x}$ is proportional to the weighted sum of square distances of each pair (x_i, x_j) with weight a_{ij} . In other words, we have the following:*

$$\bar{x}^T L \bar{x} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} (x_i - x_j)^2$$

Proof: In order to show this result, we can expand the expressions on both sides of the equation in the statement of the lemma and examine the coefficient of each term on both sides. For $i \neq j$, it can be shown that the coefficient of any term of the form $x_i x_j$ is $-(a_{ij} + a_{ji})$ on both sides. On the other hand, for any i , it can be shown that the coefficient of x_i^2 is $\frac{1}{2} \sum_{j=1}^n (a_{ij} + a_{ji})$. ■

There are several observations that one can make from the above result. These observations are used in various ways in machine learning applications on graphs:

1. An immediately obvious observation is that $\bar{x}^T L \bar{x}$ is nonnegative for any $\bar{x}$ according to Lemma 10.5.3. Therefore, it is positive semidefinite.
2. The fact that the unnormalized graph Laplacian is positive semidefinite can be easily extended to the symmetrically normalized graph Laplacian (see Problem 10.5.1). Even though the notion of positive semidefiniteness is defined only for symmetric graphs, it can also be shown that the asymmetrically normalized graph Laplacian has nonnegative eigenvalues and satisfies $\bar{x}^T L_a \bar{x} \geq 0$ for all $\bar{x}$. In other words, it satisfies an extended notion of positive semidefiniteness for asymmetric graphs. Each type of Laplacian always has one of its eigenvalues as 0, because Laplacians are singular matrices with a null space of rank at least 1. The null space has rank exactly 1 if the graph is connected. Since Laplacians always have nonnegative eigenvalues, it follows that 0 is the smallest eigenvalue, which is unique for connected graphs.
3. Finding a unit vector $\bar{x}$ that minimizes $\bar{x}^T L \bar{x}$ will find a 1-dimensional embedding of the vertices, so that vertex pairs $[i, j]$ connected by edges (i, j) of heavy weight a_{ij} are close together in terms of the value of $(x_i - x_j)^2$. This can be easily inferred from the weighted sum-of-squares interpretation of $\bar{x}^T L \bar{x}$ in Lemma 10.5.3. In other words, this type of objective function finds an embedding that is friendly to clustering, which is why it is useful for spectral clustering.

The following result can be shown relatively easily by using Lemma 10.5.3 and some properties of determinants.

Problem 10.5.1 *Let L , $L_a = \Delta^{-1} L$, and $L_s = \Delta^{-1/2} L \Delta^{-1/2}$ be the unnormalized, asymmetrically normalized, and symmetrically normalized Laplacians of an undirected graph with nonnegative weights on edges. Show the following: (i) The value of $\bar{x}^T L_s \bar{x}$ is always nonnegative for any $\bar{x}$; (ii) the eigenvalues of L_s and L_a must be the same; and (iii) The value of $\bar{x}^T L_a \bar{x}$ is always nonnegative.*

The overall approach for spectral clustering works as follows:

1. Select one of the three Laplacians L , L_a , and L_s . Find the **smallest** k eigenvectors of the Laplacian, and create an $n \times k$ matrix Q_k , whose columns contain these eigenvectors after scaling them to unit norm. Each row of Q_k corresponds to a k -dimensional embedding of a vertex.
2. In the event that the symmetric Laplacian L_s was chosen in the first step, perform the additional step of normalizing each row of Q_k to unit norm.
3. Apply a k -means algorithm to the embedding representations of the different vertices.

An important point is that we use the large eigenvectors of similarity matrices, whereas we use the *small* eigenvectors of the Laplacian. This is not surprising because of the relationship between the two. For examples, the symmetrically normalized adjacency matrix S is related to the symmetric Laplacian L_s as $L_s = I - S$. As a result, *the eigenvectors of the two matrices are identical and the corresponding eigenvalues sum to 1*. We summarize these results in terms of the equivalence of the kernel view and the Laplacian view of spectral clustering.

Lemma 10.5.4 (Equivalence of Kernel and Laplacian View) *Let A be an $n \times n$ adjacency matrix, let P be its stochastic transition matrix, and S be its symmetrically normalized adjacency matrix. Let $L_a = I - P$ and $L_s = I - S$ be the asymmetric and symmetric Laplacians, respectively. Then, the following are true:*

- *The **largest** k eigenvectors of the stochastic transition matrix P are the same as the **smallest** k eigenvectors of asymmetric Laplacian L_a .*
- *The **largest** k eigenvectors of the symmetrically normalized adjacency matrix S are the same as the **smallest** k eigenvectors of symmetric Laplacian L_s .*

Proof: We show the result in the case of the symmetric Laplacian S . The proof for the asymmetric Laplacian is similar. The eigenvector of any matrix S with eigenvalue λ is also an eigenvector of the matrix $(I - S)$ with eigenvalue $1 - \lambda$. This is because $S\bar{x} = \lambda\bar{x}$ is true if and only if $(I - S)\bar{x} = (1 - \lambda)\bar{x}$ is true. Therefore, large eigenvectors of S correspond to small eigenvectors of $L_s = (I - S)$. ■

It is noteworthy that the small eigenvectors of the *unnormalized* Laplacian L are not exactly the same as the large eigenvectors of the adjacency matrix A . In any case, the use of unnormalized variants is relatively uncommon in most practical applications.

10.5.2.2 Optimization Model with Laplacian

The fact that the small eigenvectors of the Laplacian are used for embedding can also be derived with the use of an optimization model involving the Laplacian. In fact, this approach is the popular presentation of most textbooks and surveys [84]. We recap the result from Lemma 10.5.3, according to which $\bar{x}^T L \bar{x}$ provides the weighted sum-of-square distances between pairs of data points:

$$\bar{x}^T L \bar{x} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} (x_i - x_j)^2$$

Here, $\bar{x}$ is an n -dimensional vector, which contains one coordinate for each vertex. Note that one can easily extend this result to a k -dimensional embedding of each vertex by using

k vectors $\bar{x}_1 \dots \bar{x}_k$. In this case, the n -dimensional vector $\bar{x}_i$ contains the i th coordinate of the n different vertices. In such a case, the sum over the k different values of $\bar{x}_i^T L \bar{x}_i$ provides the weighted sum-of-square Euclidean distances for the embedding. Therefore, a clustering-friendly embedding may be defined by the following optimization model:

$$\begin{aligned} & \text{Minimize } \sum_{i=1}^k \bar{x}_i^T L \bar{x}_i \\ & \text{subject to:} \\ & \|\bar{x}_i\|^2 = 1 \quad \forall i \in \{1 \dots k\} \\ & \bar{x}_1 \dots \bar{x}_k \text{ are mutually orthogonal} \end{aligned}$$

This optimization model tries to find a k -dimensional embedding of each vertex, so that the weighted sum-of-square Euclidean distances is minimized. The above model is a special case of the norm-constrained optimization problem discussed in Section 3.4.5 of Chapter 3. As discussed in Section 3.4.5, the smallest k eigenvectors of L provide a solution to this optimization problem. Note that one can use exactly the same model for the symmetrically normalized Laplacian by replacing L with L_s in the above model. The case of the asymmetric Laplacian is slightly different, because the problem boils down to the following:

$$\begin{aligned} & \text{Minimize } \sum_{i=1}^k \bar{x}_i^T L \bar{x}_i \\ & \text{subject to:} \\ & \bar{x}_1 \dots \bar{x}_k \text{ are } \Delta\text{-orthonormal} \end{aligned}$$

The only difference from the original optimization problem is that the vectors are Δ -orthonormal. The notion of Δ -orthonormality is defined as follows:

$$\bar{x}_i^T \Delta \bar{x}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

Imposing Δ -orthonormality de-emphasizes the impact of high-degree vertices. The optimal solution can be shown to be the smallest k eigenvectors of $\Delta^{-1}L$. We leave the proof of this result as a practice problem.

Problem 10.5.2 *Show that the optimum solution to the optimization model of asymmetric spectral clustering corresponds to the smallest eigenvector of $L_a = \Delta^{-1}L$.*

The key hint in the above problem is to use a variable transformation, wherein each $\bar{x}_i = \sum_{j=1}^n \beta_{ij} \bar{p}_j$ is expressed as a linear combination of the basis system of unit eigenvectors of $\Delta^{-1}L$. Subsequently, one has to solve for the coefficients β_{ij} , while transforming the objective function and the constraints in terms of these coefficient variables. Show that the eigenvectors $\bar{p}_1 \dots \bar{p}_n$ are Δ -orthogonal, and use it to simplify the objective function in terms of the different values of β_{ij} . It can be shown that all values of β_{ij} will be either $1/\sqrt{\Delta}$ or 0.

10.5.3 The Matrix Factorization View of Spectral Clustering

Spectral clustering is a form of matrix factorization. Both asymmetric and symmetric spectral clusterings are obtained using the following factorizations:

$$S = \underbrace{Q_{(s)}}_U \underbrace{\Lambda Q_{(s)}^T}_{V^T} \quad [\text{Symmetric}]$$

$$P = \underbrace{Q_{(a)}}_U \underbrace{\Lambda Q_{(a)}^{-1}}_{V^T} \quad [\text{Asymmetric}]$$

In other words, both factorizations can be expressed in the form UV^T . We can also approximately factorize either S or P into UV^T using the gradient-descent methods of Chapter 8. Note that both U and V are $n \times k$ matrices, where k is the rank of the factorization. Subsequently, the k -dimensional rows of U and V can be concatenated in order to create the $2k$ -dimensional embedded representations of each vertex. The result will not be exactly the same as that of spectral clustering, but this generalized approach will often provide similar results to spectral clustering. This generalized view is useful in cases where the direct application of spectral clustering is not possible. For example, in the case of directed graphs, the adjacency matrix may not be diagonalizable with real-valued eigenvectors and eigenvalues. In such cases, we can use generalized forms of the factorization on the directed graph.

Let A be an adjacency matrix of a directed graph. As in the case of undirected adjacency matrices, weak links are removed by keeping only those edges, which are *both* among the top- k incoming edges and the top- k outgoing edges of the two vertices at their end points. Note that the matrix A will not be symmetric either before or after removal of the weak links (since the graph is directed to begin with). Let δ_i^{in} be the weighted indegree of the i th vertex, which is obtained by adding the (possibly non-binary) elements of the i th column of A . Similarly, let δ_i^{out} be the weighted outdegree of the i th vertex, which is obtained by summing the values in the i th row of A . One can create corresponding $n \times n$ diagonal matrices denoted by Δ_{in} and Δ_{out} , whose diagonal elements are the δ_i^{in} and δ_i^{out} , respectively. Then, each edge (i, j) is normalized using the geometric mean of outdegree at i and the indegree at j :

$$a_{ij} \Leftarrow \frac{a_{ij}}{\sqrt{\delta_i^{out}} \sqrt{\delta_j^{in}}}$$

One can also write this relationship in matrix form with the use of the normalized matrix N :

$$N = \Delta_{out}^{-1/2} A \Delta_{in}^{-1/2}$$

Once the matrix N has been computed, we can factorize it as $N \approx UV^T$ using any of the methods discussed in Chapter 8. Here, U and V are $n \times k$ matrices, where k is the rank of the factorization. The i th row of U provides the *outgoing factor* of the i th vertex (or *sender factor*), and is related to some of the centrality measures discussed in the next section. The i th row of V provides the *incoming factor* of the i th vertex (or *receiver factor*). One can concatenate these representations to create a $2k$ -dimensional representation of each vertex. Subsequently, this representation can be used for clustering.

10.5.3.1 Machine Learning Application: Directed Link Prediction

One can use the aforementioned approach for directed link prediction. This type of approach can be useful for link prediction in directed follower-followee networks like Twitter. Once the

normalized matrix N has been factorized as $N \approx UV^T$, the reconstruction UV^T predicts the directed links of the adjacency matrix. Normally, link prediction is required in undirected graphs, although some applications may also require the prediction of links in directed graphs. It is important to note that the (i, j) th entry in UV^T may not be the same as the (j, i) th entry in UV^T . In other words, the prediction of links is direction-sensitive. After all, the probability of a teenager following a famous rock star on Twitter is not the same as that of the rock star following the teenager.

10.5.4 Which View of Spectral Clustering Is Most Informative?

This section provides multiple presentations of spectral clustering. In the following, we provide a broader perspective on these different views of spectral clustering:

1. **The kernel view with adjacency matrices:** The *normalized* variations of spectral clustering can be viewed as special cases of the similarity-based clustering methods with explicit feature engineering (cf. Section 9.5.1.1 of Chapter 9). The main difference between spectral clustering and this family of methods is only in terms of how the matrix is preprocessed to remove weak links. The main advantage of the kernel view is that it provides a unified view with all the other kernel methods we have seen so far in Chapter 9. A spectral method is simply an equal citizen of the vast family of kernel methods such as kernel k -means – nothing more and nothing less. The main distinguishing characteristic is the heuristic sparsening/normalization of the similarity matrix and the normalization (whitening) of engineered features.
2. **The Laplacian view:** The use of the Laplacian is the dominant treatment in popular expositions of spectral clustering [84]. Because of the different nature of this treatment as compared to kernel or factorization methods, spectral clustering is often viewed in a very different light than other related embedding methods. The Laplacian view is often interpreted as an elegant *discrete* optimization problem of finding minimum cuts in a normalized graph [115]. However, the actual optimization problem of spectral clustering is only a *continuous approximation* of this problem, which could be an arbitrarily poor approximation of the discrete version.
3. **The matrix factorization view and its extensions:** Asymmetric forms of spectral clustering can be viewed in the context of matrix factorization. Most importantly, this point of view provides a generalization of the approach to clustering *directed* graphs, which is not possible with the vanilla version of spectral clustering.

An important principle in understanding a method well is to understand its relationship with other similar methods. This type of understanding promotes useful extensions and applications. Spectral clustering is simply *a special case of a kernel clustering method with explicit feature engineering and a data-dependent kernel*. Because of its need to postprocess the features extracted from the kernel before applying k -means on the features, it cannot be used in conjunction with the kernel trick (like kernel k -means).

10.6 The Left Eigenvectors of Graph Matrices

As discussed in the previous section, the right eigenvectors of the stochastic transition matrix of the adjacency matrix A are used to create the Shi-Malik embedding for clustering. In this section, we will explore the left eigenvectors of the stochastic transition matrix.

However, there are several differences from the exposition in the previous section. First, we will examine the characteristics of only the *principal* (i.e., largest) eigenvector, which has an eigenvalue of 1 according to the Perron-Frobenius theorem. Second, since we will be examining applications associated with the graph structure of the Web (rather than clustering), the focus will be on directed graphs rather than undirected graphs. After all, Web page linkage structure is not symmetric. Finally, the clustering application discussed in the previous section always modifies the adjacency matrix in order to remove weak links. This modification is not made for the applications discussed in this section. Rather a different type of modification is used, which focuses on making the directed graph strongly connected.

In many applications such as the Web, one is looking for vertices with a high level of *prestige*. Intuitively, a Web page has a high level of prestige if many Web pages point to it. However, simply using the number of Web pages pointing to a page might be a deceptive indicator of its prestige, because the pages pointing to it might be of low quality themselves. Therefore, one typically wants to discover pages that are pointed to by other high prestige pages. One can model this type of recursive relationship by using the notion of *random walks* in the graph.

Imagine a directed graph with adjacency matrix A , weighted degree matrix Δ , and stochastic transition matrix $P = \Delta^{-1}A$. One can interpret each entry p_{ij} in P as the probability of a transition of random surfer from Web page i to Web page j , under the assumption that the surfer selects the vertices outgoing from i using the vector of transition probabilities $[p_{i1}, p_{i2}, \dots, p_{in}]$. Note that these transition probabilities sum to 1. The *PageRank*-based prestige of a vertex (Web page) is defined as the steady-state probability of a random surfer visiting that vertex (Web page). Interestingly, the process that we just described is precisely the transition process of a *Markov chain*. A Markov chain consists of a set of states (vertices) along with a set of transitions (edges with probabilities) among them. Therefore, a Markov chain is perfectly described by the graph structures introduced in this section. Furthermore, a Markov chain has a well-defined procedure for finding steady state probabilities in terms of eigenvectors of its transition matrix.

One question that arises is as to the conditions under which a Markov chain has steady-state probabilities that are independent of the starting vertex of the random surfer. Such Markov chains are referred to as *ergodic* and they must satisfy the following condition:

Definition 10.6.1 (Ergodic Markov Chain) *A Markov chain is defined as ergodic, if its transition matrix is strongly connected.*

Graphs that are not strongly connected might have steady-state probabilities for a walk that depend on the starting point of the walk. For example, consider the directed graph in Figure 10.5(b), which is not strongly connected. In this case, starting a random walk at vertex 1 will result in steady-state probabilities that are distributed only among the vertices in the component A1. However, starting the walk at vertex 6 could lead to either of the components A1 or A2.

An immediate problem that arises because of this requirement is that many real applications do not result in directed graphs that are strongly connected. For example, the Web is certainly not a directed graph that is strongly connected. When you set up a new Website, the chances are that you might point to many other Websites, but no one might know of your Website or point to it. This can be a problem in terms of computing steady-state probabilities.

This problem is solved with the use of *restart probabilities*. In each step, the random surfer is allowed to reset to a completely random vertex in the network with probability

$\alpha < 1$ and continue with the random walk with probability $(1 - \alpha)$. The value of α is a hyper-parameter that is chosen in an application-specific manner. The transition matrix P is obtained from the old transition matrix P_o as follows:

$$P \Leftarrow (1 - \alpha)P_o + \alpha M/n$$

Here, M is an $n \times n$ matrix containing 1s. The matrix M/n is a transition matrix in which one can move from any vertex to another with probability $1/n$. Therefore, this is a strongly-connected restart matrix. The final transition matrix P is a weighted combination of the original transition matrix and the restart matrix. Henceforth, we will always assume that the transition matrix is strongly connected, and the restart matrix has been incorporated as a preprocessing step.

10.6.1 PageRank as Left Eigenvector of Transition Matrix

Let π_i be the steady-state probability of vertex i under the random walk model. Then, the probability π_i of visiting a particular vertex i is given by the sum of the probabilities of transitioning into that vertex from each of its incoming vertices. The probability of transitioning into vertex i from vertex j is given by $\pi_j p_{ji}$. We can write this relationship mathematically for each vertex $i \in \{1, \dots, n\}$ as follows:

$$\pi_i = \sum_{j=1}^n \pi_j p_{ji} \quad \forall i \in \{1 \dots n\}$$

Note that we have n equations in n variables $\pi_1 \dots \pi_n$. We can denote the vector of n variables by $\bar{\pi} = [\pi_1, \pi_2, \dots, \pi_n]^T$. The above system of equations can then be written in vector form as follows:

$$\bar{\pi}^T = \bar{\pi}^T P$$

This is exactly a left-eigenvector equation at an eigenvalue of 1. From the Perron-Frobenius theorem, we already know that a unique eigenvector with eigenvalue 1 exists for the stochastic transition matrices associated with strongly connected graphs.

One can use the power method (cf. Section 3.5.2 of Chapter 3) to solve for $\bar{\pi}$ since it is the principal eigenvector of the stochastic transition matrix. The approach works by initializing $\bar{\pi}$ to a vector of random positive values between 0 and 1, and then scaling all values to sum to 1. Subsequently, the following iterative process is repeated:

1. $\bar{\pi}^T \Leftarrow \bar{\pi}^T P$
2. Normalize $\bar{\pi}$ so that its elements sum to 1.

This approach is applied to convergence.

Problem 10.6.1 (Efficient Computation) *One problem with the use of restart is that it makes the transition matrix P dense, even when the underlying graph is sparse. Discuss how you can treat the restart component of P more carefully, so as to be able to compute $\bar{\pi}$ using only sparse matrix operations.*

A hint for solving the above problem is that the final transition matrix can be represented in terms of the original transition matrix P_o as $(1 - \alpha)P_o + \alpha \bar{1}_n \bar{1}_n^T/n$. Here, $\bar{1}_n$ is an n -dimensional column vector of 1s.

10.6.2 Related Measures of Prestige and Centrality

The *PageRank* algorithm can be easily applied to undirected networks by replacing each undirected edge with two directed edges in opposite directions. When applied to directed networks, *PageRank* is a *prestige* measure, and when applied to undirected networks, *PageRank* is a *centrality measure*. Prestige computes an asymmetric measure of importance, and is high when a vertex is reachable from other vertices. However, if a vertex points to many vertices but is reachable from very few vertices, its *PageRank* might be quite low. Because of the symmetric nature of undirected networks, incoming and outgoing reachability cannot be distinguished. Therefore, it is considered a centrality measure. The *PageRank* is only one of many mechanisms used for centrality and prestige computation in networks. The first among them is *eigenvector centrality*, which uses the principal left eigenvector of the (unnormalized) adjacency matrix, as opposed to the stochastic transition matrix:

Definition 10.6.2 (Eigenvector Centrality) *Let A be the $n \times n$ adjacency matrix of an undirected graph. Then, the n components of the principal left eigenvector define the eigenvector centrality of the vertices. In other words, the eigenvector centrality of each of the n vertices is contained in the n -dimensional vector $\bar{\pi}^T A = \lambda \bar{\pi}^T$. Here, λ is the largest eigenvalue of A .*

According to the Perron-Frobenius theorem, the largest eigenvalue lies between the maximum and average degree of the matrix. Like *PageRank*, eigenvector centrality can be computed using the power method. For directed graphs, one can also compute the notion of *eigenvector prestige*.

Definition 10.6.3 (Eigenvector Prestige) *Let A be the $n \times n$ adjacency matrix of a directed graph. Then, the n components of the principal left eigenvector define the eigenvector prestige of the vertices. In other words, the eigenvector prestige of each of the n vertices is contained in the n -dimensional vector $\bar{\pi}^T A = \lambda \bar{\pi}^T$. Here, λ is the largest eigenvalue of A .*

As in the case of *PageRank*, the largest eigenvector may not be unique if the graph is not strongly connected. Therefore, the matrix A may need to be averaged with a restart matrix M/n in order to make it strongly connected. Here, M is a matrix of 1s. As in *PageRank*, the averaging is done in a weighted way with smoothing parameter α .

Finally, it is interesting to explore the left eigenvectors of symmetrically normalized matrices. Let A be the adjacency matrix of an undirected network, and Δ be its weighted degree matrix in which the i th diagonal entry is the degree δ_i . Then, the symmetrically normalized matrix is given by the following:

$$S = \Delta^{-1/2} A \Delta^{-1/2}$$

Since the matrix S is symmetric, its left and right eigenvectors are the same. The principal eigenvector of this matrix contains the square-roots of the vertex degrees. This result is given in Corollary 10.5.1. The degree of a vertex is its *degree centrality*.

Since degree centrality can be computed trivially from the adjacency matrix, it does not make sense to use eigenvector computation. Nevertheless, it is interesting that all eigenvectors relate in one way or another to centrality measures. The corresponding notion of *degree prestige* is defined by the indegree of each vertex in a directed graph. It is possible to relate degree prestige to the eigenvectors of the adjacency matrix after careful normalization, although it is not practically useful. Importantly, the normalization can be done only with weighted indegrees (i.e., column sums) of A rather than outdegrees (i.e., row sums). Note

Table 10.1: Relationship of different eigenvectors to prestige and centrality measures

	Centrality (undirected)	Prestige (directed)
Unnormalized matrix A	Eigenvector Centrality $\lambda \bar{\pi}^T = \bar{\pi}^T A$	Eigenvector Prestige $\lambda \bar{\pi}^T = \bar{\pi}^T A$
Left-normalized matrix $P = \Delta^{-1} A$	PageRank Centrality $\bar{\pi}^T = \bar{\pi}^T P$	PageRank Prestige $\bar{\pi}^T = \bar{\pi}^T P$
Bi-normalized matrix Undirected: $S = \Delta^{-1/2} A \Delta^{-1/2}$ Directed: $S_d = \Delta_{in}^{-1/2} A \Delta_{in}^{-1/2}$	Degree Centrality $\bar{\pi}^T = \bar{\pi}^T S$	Degree Prestige $\bar{\pi}^T = \bar{\pi}^T S_d$

that S will not be symmetric, because A is not symmetric to begin with. Just as directed graphs associate *prestige* measures with vertex *indegrees*, it is also possible to associate *gregariousness* measures with vertex *outdegrees*. Intuitively, the gregariousness of a vertex is defined by the propensity of that vertex to easily reach other vertices via directed paths. This is a complementary idea to prestige measures, in which vertices with high prestige are likely to be easily reached by other vertices via directed paths. In other words, the key difference is in terms of the direction of the edges in the two cases. We leave the modeling of this problem as a practice exercise.

Problem 10.6.2 (Gregariousness) *Each of the three prestige measures (i.e., eigenvector, PageRank, and degree) has a corresponding gregariousness measure. The difference is that prestige measures value in-linking vertices, whereas gregariousness measures value out-linking vertices. For example, the degree gregariousness is the outdegree of a vertex. How would you define the different gregariousness measures of a directed graph in terms of the eigenvectors of an appropriately chosen matrix? This matrix should be defined as a function of the (directed) adjacency matrix A , outdegree matrix Δ_{out} , and indegree matrix Δ_{in} .*

10.6.3 Application of Left Eigenvectors to Link Prediction

The left eigenvector of a stochastic transition matrix can also be used for link prediction. In link prediction, we attempt to find pairs of vertices that do not have a link between them, but are likely to become connected in the future (based on the fact that these vertices are connected to similar vertices). In the following, we assume the case of undirected link prediction although the approach can also be used for directed link prediction (cf. Problem 10.6.3). Given a vertex i , we would like to find all vertices to which it is likely to become connected in the future. The main difference from *PageRank* is in terms of how the restart probabilities are set. Instead of restarting at any vertex, we allow a restart at only a vertex of interest i . Therefore, we first compute an $n \times n$ restart matrix $R = [r_{ij}]$. The matrix R is also a stochastic transition matrix in which all entries in the i th column are set to 1s and the remaining entries are 0s. Then, the current transition matrix P of the undirected graph is modified as follows with the use of restart probability $\alpha \in (0, 1)$:

$$P \Leftarrow (1 - \alpha)P + \alpha R$$

Subsequently, the components of the left eigenvectors of P provide *personalized PageRank* values for vertex i . The value of α provides the trade-off between the level of personalization and the core social popularity of a vertex.

Problem 10.6.3 (Directed Link Prediction) *Propose a method for using the left eigenvectors of appropriately chosen matrices to predict (i) incoming links, and (ii) outgoing links, of a given vertex.*

10.7 Eigenvectors of Reducible Matrices

The Perron-Frobenius theorem incorporates a strong connectivity assumption. This section examines cases where this assumption does not hold.

10.7.1 Undirected Graphs

Consider an undirected graph with an $n \times n$ adjacency matrix A , degree matrix Δ , and stochastic transition matrix $P = \Delta^{-1}A$. If the undirected graph is not connected, its adjacency matrix and stochastic transition matrix can both be represented in block diagonal form. Specifically, the stochastic transition matrix of a graph containing two connected components can be represented as follows:

$$P = \begin{bmatrix} P_{11} & 0 \\ 0 & P_{22} \end{bmatrix}$$

In general, a graph having r connected components will have r square blocks along the diagonal. Each block P_{ii} along the diagonal is a stochastic transition matrix within that subset of vertices.

The Perron-Frobenius theorem states that the largest eigenvector of the stochastic transition matrix of a connected graph is unique, and has an eigenvalue of 1. The uniqueness of the largest eigenvector of the adjacency matrix no longer holds when the graph is not connected. For example, in the case of the two-component adjacency matrix, an eigenvector containing 1s for the first component and 0s for the second component has an eigenvalue of 1. This is because the block P_{11} is a transition matrix in its own right, and it is the only part of P that will interact with the non-zero part of the eigenvector. Similarly, one can create an eigenvector containing 0s for the first component and 1s for the second component. This eigenvector also has an eigenvalue of 1. In other words, the dimensionality of the *eigenspace* corresponding to an eigenvalue of 1 is equal to the number of connected components. The eigenvectors corresponding to different connected components form a basis of this eigenspace.

Property 10.7.1 *The dimensionality of the vector space corresponding to the largest eigenvectors (i.e., eigenvectors with eigenvalue 1) of the stochastic transition matrix of an undirected graph is equal to the number of connected components in it.*

We further note that one can compute the number of connected components in a graph using linear algebra.

Property 10.7.2 *Let P be the stochastic transition matrix of an undirected graph. Then, the dimensionality of the null space of $(P - I)$ yields the number of connected components in the graph.*

It is evident from Property 10.7.1 that the eigenspace of the matrix P (for the eigenvalue of 1) is the null space of $(P - I)$. The null space of a matrix can be easily computed using SVD.

10.7.2 Directed Graphs

The notion of connectivity is more complex in directed graphs, because the direction of an edge matters in the definition of concepts like strong connectivity. As in the case of undirected graphs, we will assume that we are working with the stochastic transition matrix.

First, the creation of a stochastic transition matrix from a directed adjacency matrix is not always possible because some vertices might not have any *outgoing* edges. In other words, the corresponding row in the adjacency matrix contains 0s, although its column does contain non-zero entries. It is impossible to normalize such rows to sum to 1. Therefore, the following analysis will assume that such dead-end nodes do not exist. When such dead-end nodes do exist, one can add a self-loop with probability of 1 to facilitate the following analysis. This type of addition of self-loops is often performed in numerous machine learning algorithms like *PageRank* and collective classification (in the presence of dead-end nodes). Another assumption that we make to facilitate analysis is that the graph is fully connected if the directions of edges are ignored.

When a graph is not strongly connected, the matrix will be *block upper-triangular*, in which the diagonal contains square blocks of strongly connected components with some additional non-zero entries above these diagonal blocks. These additional entries only allow unidirectional walks between blocks (sets) of vertices. It is easiest to understand this type of block structure in terms of random walks. Strongly connected graphs result in *ergodic Markov chains* in which the steady-state probabilities do not depend on where the random walk is started. Furthermore, all vertices have non-zero probability of being reached in steady state.

Graphs that are not strongly connected do not satisfy these properties. Such graphs containing vertices of two types:

1. The first type is the *transient vertex set* in the graph. The transient vertex set is a maximal set of vertices, such that the direction of the edges between vertices belonging to the transient set and all other vertices are always outgoing from vertices belonging to the transient set. Note that the vertices in the transient set may or may not be connected/strongly connected to one another, and therefore the edge structure on the subgraph of transient vertices may be arbitrary. These vertices are referred to as transient because a random walk will never visit these vertices in steady-state. Once a walk exits this component, it is no longer possible for the walk to return to this component in steady state. For example, in the case of Figure 10.5(a), vertex 9 is the only transient vertex. On the other hand, in Figure 10.5(b), the vertices 1, 4, 5, 6, and 7 are transient vertices. The transient vertices are labeled by ‘T’ in these figures. The reader should take a moment to verify that a random walk starting at any of these vertices will eventually reach a vertex outside this set so that it becomes impossible to ever visit any of these vertices again. The transient component is an essential property of reducible graphs. If a connected graph does not have a transient component, then it is also strongly connected.

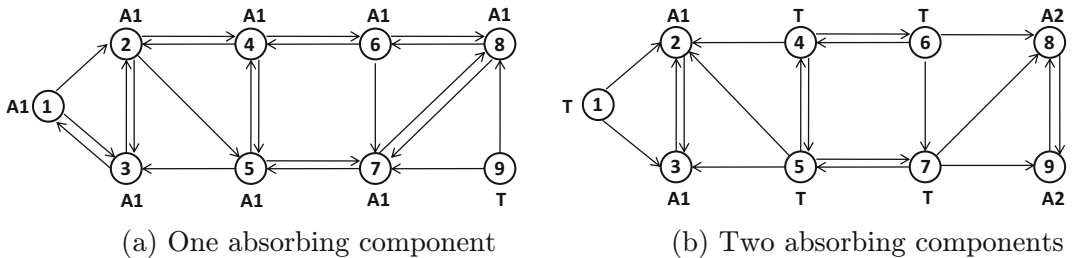


Figure 10.5: Examples of directed graphs that are not strongly connected. Transient vertices are labeled as ‘T’ and vertices belonging to absorbing components are labeled as ‘A1’ and ‘A2’

2. In addition to the transient vertex set, the graph contains l vertex-disjoint components, which are referred to as *absorbing components*. Each absorbing component is strongly connected in terms of the subgraph induced by its vertex set. Furthermore, an absorbing component only has incoming edges from transient vertices, is not connected to other absorbing components, and has no outgoing edges. Each absorbing component has a non-zero probability to be visited in steady-state from a random walk that starts at a random vertex in the network. However, the steady-state probability depends on the vertex at which the random walk starts. In Figure 10.5(a), there is a single absorbing component containing all vertices except vertex 9. On the other hand, Figure 10.5(b) contains two absorbing components, which are labeled by ‘A1’ and ‘A2.’ Note that starting the random walk at vertex 1 will always reach absorbing component A1, but will never reach absorbing component A2. Starting the walk at vertex 4 will allow both A1 and A2 to be reached.

Without loss of generality, we assume that the vertices of such a graph are ordered as follows. The first block of this graph contains all the transient vertices. For this set a_{ij} can be non-zero for each transient vertex i and j can be any value from 1 to n . All other l absorbing components are arranged in block diagonal form, as in the case of undirected matrices. For example, a graph with a transient set and three absorbing components will have the following block structure of the stochastic transition matrix on appropriate reordering the vertex indices (to put the transient vertices first and the vertices of the absorbing components contiguously in succession):

$$P = \begin{bmatrix} P_{11} & P_{12} & P_{13} & P_{14} \\ 0 & P_{22} & 0 & 0 \\ 0 & 0 & P_{33} & 0 \\ 0 & 0 & 0 & P_{44} \end{bmatrix}$$

It is important to note that the square blocks P_{22} , P_{33} , and P_{44} correspond to the edges within absorbing components, and are complete stochastic transition matrices. This matrix has three eigenvectors in the eigenspace belonging to eigenvalue 1. Each of these eigenvectors can be defined by an absorbing component. It is easier to discuss the left eigenvectors. First one can compute the principal left eigenvectors of each of the blocks P_{22} , P_{33} , and P_{44} (corresponding to the absorbing components) separately. Each such eigenvector of the absorbing submatrix can be used to define a left eigenvector of P by setting the remaining components of the larger eigenvector to zeros. Therefore, the Markov chains defined by such reducible graphs have more than one solution to the steady-state equation $\bar{\pi}^T P = \bar{\pi}^T$.

Although it is possible for directed graphs to be completely disconnected (i.e., disconnected irrespective of the direction of edges), this situation rarely arises in machine learning applications. In this chapter, we ignore this case, because it is not very interesting from an application-centric point of view. All connected graphs that are not reducible will have at least some transient vertices that have a one-way connection to strongly connected components. The basic structure of a reducible graph always appears in the form of Figure 10.6, assuming that the graph is not completely disconnected. Note that the transient vertices need not be strongly connected (or even connected) when considered as a subgraph. For example, the transient vertices in Figure 10.5(b) (which are labeled by ‘T’) are not connected to one another, when considered as a subgraph. Each of the satellite components in Figure 10.6 is an absorbing component. If a random walk is performed on the full graph, and the random walk happens to enter an absorbing component, the walk will never exit the component. It is possible for a reducible network to contain only one absorbing component,

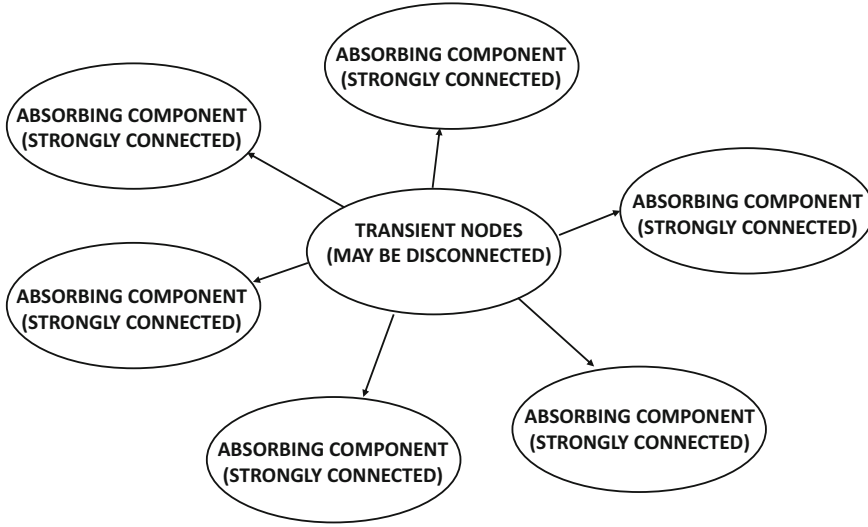


Figure 10.6: If a graph is not strongly connected, it will have at least some transient vertices with one-way connections to strongly connected components

as long as some of the vertices are transient. In fact, an adjacency matrix for a directed, connected graph is reducible if and only if transient vertices exist in the graph.

We provide a number of extended properties of these types of reducible matrices. Let P be the $n \times n$ stochastic transition matrix of a connected (but not strongly connected), directed graph with at least one transient vertex and l absorbing components. We provide an intuitive interpretation of the principal eigenspace in P :

1. The total rank of the eigenspace corresponding to the principal eigenvalue of 1 is always l , where l is the number of absorbing components.
2. It is possible to construct an eigenspace basis of the principal left eigenvectors, so that each eigenvector “belongs” to an absorbing component. The i th component of the left eigenvector belonging to the j th absorbing component contains the steady-state probability of vertex i in a random walk starting in the j th absorbing component.
3. It is possible to construct an eigenspace basis of principal right eigenvectors, so that each eigenvector “belongs” to an absorbing component. The i th component of the right eigenvector belonging to the j th absorbing component contains the probability that a walk starting at vertex i will terminate in absorbing component j . This probability can be fractional for transient nodes and is 1 for nodes belonging to component j . Note that the sum of these l eigenvectors is itself an eigenvector of $1s$ (which is what we obtain in the irreducible case). This basis of l eigenvectors is very useful in applications like *collective classification* (see next section and Problem 10.8.1).

10.8 Machine Learning Applications

In this section, we will discuss a number of machine learning applications of graphs.

10.8.1 Application to Vertex Classification

Reducible matrices can be used for applications to vertex classification in graphs. The problem of vertex classification is also referred to as *collective classification*. The problem of collective classification is suited to undirected graphs with a symmetric adjacency matrix A . In collective classification, a subset of the vertices are labeled with one of k labels, denoted by $\{1, \dots, k\}$. Some of the vertices may not be labeled. The goal is to classify the unlabeled vertices based on the structure of the graph and the known labels. This problem often arises in social networks, where one attempts to find actors with particular properties (e.g., interest in a particular product), based on other (known) actors with these properties. The known actors can, therefore, be labeled with their properties. An example of an undirected graph with a subset of vertices that are labeled either 'A' or 'B' is shown in Figure 10.7(a). Therefore, this particular example corresponds to the binary label setting with $k = 2$. For ease in discussion, assume that the edge weights of the adjacency matrix A are binary. Many vertices are not labeled in Figure 10.7(a), and the goal is to classify precisely these vertices.

The basic principle for solving this problem is to use the principle of *homophily* in social networks. The idea is that vertices tend to be connected with vertices that have similar properties. Therefore, a random walk starting at an unlabeled vertex is more likely to first reach a labeled vertex, whose label value matches its own. Therefore, a probabilistic approach for solving this problem is as follows:

Given an unlabeled vertex, perform a random walk by using the stochastic transition matrix of the adjacency matrix until a labeled vertex is reached. Output the observed label of the destination (labeled) vertex as the predicted class label of the source (unlabeled) vertex from which the random walk begins.

For better robustness, one can compute the *probability* that a vertex of each class is reached. The intuition for this approach is that the walk is more likely to terminate at labeled vertices in the proximity of the starting vertex i . Therefore, when many vertices of a particular class are located in its proximity, then the vertex i is more likely to be labeled with that class. In the particular case of Figure 10.7(a), any random walk starting from test vertex X will always reach label 'A' first rather than label 'B' because of the topology of the graph. However, it is also possible to select test vertices, where there is a non-zero probability of reaching either the label 'A' or the label 'B.' For example, if one starts the random walk at test vertex Y, then a vertex corresponding to either label 'A' or label 'B' could be reached first.

An important assumption is that the graph needs to be *label connected*. In other words, every unlabeled vertex needs to be able to reach a labeled vertex in the random walk. For undirected graphs, this means that every connected component of the graph needs to contain at least one labeled vertex. In the following discussion, it will be assumed that the entire undirected graph is connected; any undirected graph with only one connected component will always lead to a modified transition matrix that is label connected.

Since the approach is based on random walks, the first step is to create the (directed) stochastic transition matrix P from the undirected $n \times n$ adjacency matrix A . Let Δ be the (diagonal) degree matrix that contains the weighted degree $\delta_i = \sum_j a_{ij} = \sum_j a_{ji}$ on the i th entry of its main diagonal. As in the case of all other applications of this chapter, the adjacency matrix is converted into the stochastic transition matrix by using left-normalization:

$$P = \Delta^{-1}A \quad (10.7)$$

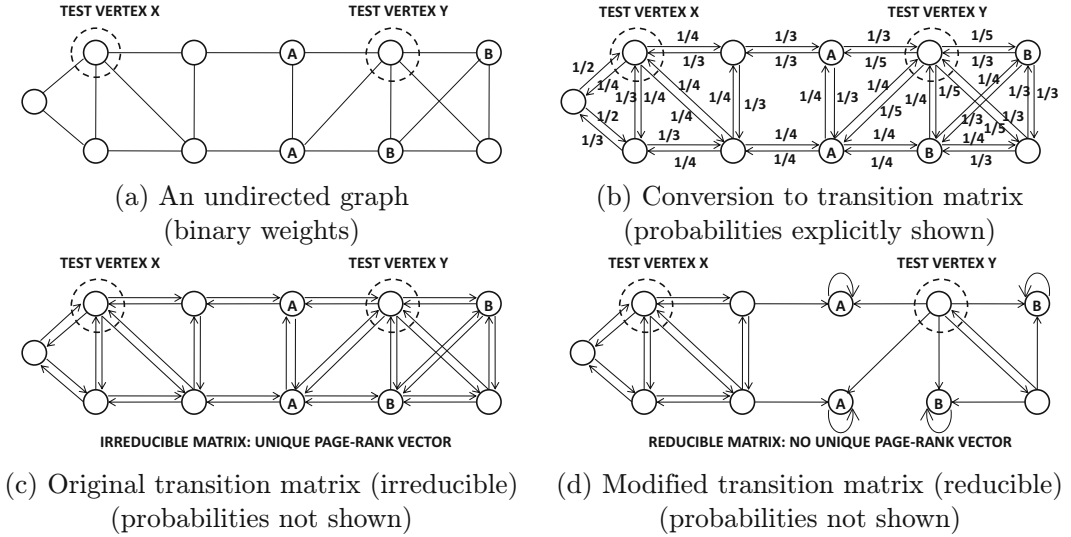


Figure 10.7: Creating directed transition graphs from undirected graph

Since this transition matrix is derived from an undirected graph, it will always be an irreducible matrix. The corresponding strongly connected graph is illustrated in Figure 10.7(b), and the probabilities on the various edges are explicitly shown. The same illustration is shown in Figure 10.7(c), but without the probabilities on the edges (to avoid clutter).

Although we can use this transition matrix to model the random walks in the graph, such an approach will not provide the *first stopping point* of the walk. In other words, we need to model the random walks in such a way that they always terminate at their *first arrival* at labeled vertices. This can be achieved by removing outgoing edges from labeled vertices and replacing them with self-loops. This results in a singleton absorbing component containing only one vertex, which we refer to as an *absorbing vertex*. Such vertices are referred to as absorbing vertices because they trap the random walk after an incoming transition. The goal of creating such absorbing vertices is to ensure that a random walk is trapped by the first labeled node it reaches.

The stochastic transition matrix P needs to be modified to account for the effect of absorbing vertices. For each absorbing vertex i , the i th row of P is replaced with the i th row of the identity matrix. Henceforth, we will assume that the matrix denoted by notation P incorporates this modification (and is therefore not exactly equal to $\Delta^{-1}A$ according to Equation 10.7). An example of the final transition graph is illustrated in Figure 10.7(d). The resulting matrix is no longer irreducible because the resulting graph is no longer strongly connected; no other vertex can be reached from an absorbing vertex. Note that this graph has exactly the structure of Figure 10.6 because each of the absorbing components is a singleton (labeled) vertex, and all unlabeled vertices are transient. Since the transition matrix P is reducible, it does not have a unique eigenvector with eigenvalue 1. Rather, it has as many principal eigenvectors with eigenvalue 1 as the number of absorbing vertices.

For any given starting vertex i , the steady-state probability distribution has positive values only at labeled vertices. This is because a random walk will eventually reach an absorbing vertex in a label-connected graph, and it will never emerge from that vertex. Therefore, if one can estimate the steady-state probability distribution of labeled nodes for

a starting unlabeled vertex i , then the probability values of the labeled vertices in each class can be aggregated. The class with the highest probability is reported as the relevant label of the unlabeled vertex i .

Note that the (i, j) th entry of P^r yields the probability of a random walk of length r starting at any vertex i to terminate at any vertex j . Because of the self-loops at absorbing vertices, all walks of length less than r are automatically included in the probability (as the remaining steps can be completed inside the self-loop). Therefore, P^∞ is the steady-state matrix of probabilities, in which the (i, j) th entry of P^∞ provides the probability that a walk starting a vertex i terminates at vertex j . For each row, we would like to aggregate the probabilities of the classes belonging to the different labels. As we will see below, this can be achieved with a simple matrix multiplication.

Let Y be an $n \times k$ matrix in which the (i, c) th entry is 1, if the i th vertex is labeled and it belongs to the class $c \in \{1, \dots, k\}$. Then, the aggregation of the probabilities of labeled vertices in each row of P^∞ is given by the matrix $P^\infty Y$. In other words, we can obtain an $n \times k$ matrix Z of probabilities of the classes for the various vertices as follows:

$$Z = P^\infty Y \quad (10.8)$$

The class with the maximum probability in Z for unlabeled vertex (row) i may be reported as its class label. This approach is also referred to as the *rendezvous approach* to label propagation [9]. How is P^∞ computed? One possibility is keep multiplying P with itself in order to compute P^∞ . One can use eigendecomposition tricks to speed up the process. However, we do not want to work with large matrices (like P^∞) of size $n \times n$. Therefore, a more efficient (but equivalent) approach is *iterative label propagation* [143].

In iterative label propagation, we initialize $Z^{(0)} = Y$ and then repeatedly use the following update for increasing value of iteration index t :

$$Z^{(t+1)} = PZ^{(t)} \quad (10.9)$$

It is easy to see that $Z^{(\infty)}$ is the same as the value of Z in Equation 10.8. Furthermore, each column of Z is a principal right eigenvector of P at convergence. Each principal eigenvector corresponds to a label with multiple absorbing components rather than a single absorbing component. Furthermore, the absorbing components are singleton vertices in this case. The following exercise generalizes this idea to finding principal eigenvectors of absorbing components with no special structure.

Problem 10.8.1 (Right Principal Eigenvectors) Consider a directed graph with l absorbing components, which are already demarcated. Discuss how you can use the ideas in iterative label propagation to create a basis of l principal right eigenvectors satisfying the following property. The i th component of the j th principal eigenvector should be equal to the probability that a walk starting at node i ends in absorbing component j .

10.8.2 Applications to Multidimensional Data

Graphs can also be applied to various machine learning applications on multidimensional data. This is because any type of multidimensional data set can be modeled as a *similarity graph*. Consider a multidimensional data set with n data instances denoted by $\bar{X}_1 \dots \bar{X}_n$. The first step is to compute the k -nearest neighbors of each data point. The similarity between any pair of points $\bar{X}_i$ and $\bar{X}_j$ can be computed with the use of a Gaussian kernel:

$$K(\bar{X}_i, \bar{X}_j) = e^{-\|\bar{X}_i - \bar{X}_j\|^2 / (2 \cdot \sigma^2)} \quad (10.10)$$

Here, σ is the bandwidth of the Gaussian kernel (cf. Table 9.1 of Chapter 9). For supervised applications, the value of σ is often tuned using out-of-sample data. For unsupervised applications, the value of σ is chosen to be of the order of the median of all pairwise Euclidean distances between points.

A graph is constructed, where the i th vertex corresponds to the data point $\bar{X}_i$. The similarity value can be used to compute the mutual k -nearest neighbor graph, as discussed in Section 10.5. The weight of an edge is set to the kernel similarity value introduced in Equation 10.10. Subsequently any of the applications such as clustering or classification (discussed in this chapter) can be used on this graph. In the case of classification, the graph is constructed on both the labeled and the unlabeled vertices. Therefore, unlike traditional classification, the classification benefits from unlabeled samples in the data. This type of approach can be helpful when the number of labeled instances is small. The use of unlabeled instances for better classification is referred to as *semi-supervised classification*. Furthermore, one can use this approach not only for multidimensional data, but for any type of data where a similarity function can be computed between the objects. After all the various forms of graph embeddings (e.g., spectral embeddings) can be viewed as special cases of kernel methods. Like any kernel method, a similarity graph and its embeddings can be used for any data type.

10.9 Summary

This chapter discusses the linear algebra of graphs. Many important structural properties of graphs such as the walks between vertices and connectivity can be inferred from the linear algebra of the adjacency matrix. The fundamental result underpinning the primary results on spectral analysis of graphs is the Perron-Frobenius theorem. A particular type of matrix associated with graphs is the stochastic transition matrix, whose principal eigenvector has an eigenvalue of 1. The dominant left eigenvector and the dominant right eigenvector(s) of the stochastic transition matrix have different types of applications. The dominant left eigenvector is used for ranking, whereas the right eigenvectors are used for clustering. One can also use a symmetrically normalized adjacency matrix for clustering, which is roughly equivalent to the use of kernel methods. Different forms of normalization of the adjacency matrix help in extracting different centrality and prestige measures. One can also compute the eigenvectors of modified versions of stochastic transition matrices in order to perform collective classification of graphs. All graph-based machine learning applications can be generalized to arbitrary data types with the construction of suitable similarity graphs over the underlying objects.

10.10 Further Reading

The basics of spectral graph theory are available in [25, 29]. A more application-centric approach may be found in [40]. The basics of graph Laplacians and spectral clustering may be found in the well-known survey by Luxburg [84]. The symmetric and asymmetric variations of spectral clustering were proposed in [98, 115]. Both the variants are roughly equivalent, although the symmetric variant is easier to relate to a kernel method. The original *PageRank* algorithm was proposed in [24]. A discussion of different types of centrality measures may be found in [138]. Different presentations of the label propagation algorithm are provided in [9, 143]. Surveys on vertex classification in graphs may be found in [17, 82].

10.11 Exercises

1. Consider the $n \times n$ adjacency matrices A_1 and A_2 of two graphs. Suppose that the graphs are known to be *isomorphic*. Two graphs are said to be isomorphic, if one graph can be obtained from the other by reordering its vertices. Show that isomorphic graphs have the same eigenvalues. [Hint: What is the nature of the relationship between their adjacency matrices in algebraic form? You may introduce any new matrices as needed.]
2. Suppose that you were given the eigenvectors and eigenvalues of the stochastic transition matrix P of an undirected graph. Discuss how you can quickly compute P^∞ using these eigenvectors and eigenvalues.
3. Let Δ be the weighted degree of matrix of an $n \times n$ (undirected) adjacency matrix A , and $\bar{e}_1 \dots \bar{e}_n$ be the n eigenvectors of the stochastic transition matrix $P = \Delta^{-1}A$. Show that any pair of eigenvectors $\bar{e}_i$ and $\bar{e}_j$ are Δ -orthogonal. In other words, any pair of eigenvectors $\bar{e}_i$ and $\bar{e}_j$ must satisfy the following:

$$\bar{e}_i^T \Delta \bar{e}_j = 0$$

4. Show that all eigenvectors (other than the first eigenvector) of the stochastic transition matrix of a connected, undirected graph will have both positive and negative components.
5. Consider the adjacency matrix A of an $n \times n$ undirected graph, which is also bipartite. In a bipartite graph, the n vertices can be divided into two vertex sets V_1 and V_2 of respectively n_1 and n_2 vertices, so that all edges occur between vertices of V_1 and vertices of V_2 . The adjacency matrix of such a graph always has the following form for an $n_1 \times n_2$ matrix B :

$$A = \begin{bmatrix} 0 & B \\ B^T & 0 \end{bmatrix}$$

Even though A is symmetric, B might not be symmetric. Given the eigenvectors and eigenvalues of A , show how you can perform the SVD of B quickly (and vice versa).

6. A complete directed graph is defined on n vertices and it contains all $n(n-1)$ possible edges in both directions between each pair of vertices (other than self-loops). Each edge weight is 1.
 - (a) Give a short reason why all eigenvalues must be real.
 - (b) Give a short reason why the eigenvalues must sum to 0.
 - (c) Show that this graph has one eigenvalue of $(n-1)$ and $(n-1)$ eigenvalues are -1 . [Express adjacency matrix as $\bar{\mathbf{1}}\bar{\mathbf{1}}^T - I$.]
7. A complete bipartite graph (see Exercise 5) is defined on 4 vertices, where 2 vertices are contained in each partition. A edge of weight 1 exists in both directions between each pair of vertices drawn from the two partitions. Find the eigenvalues of this graph. Can you generalize this result to the case of a complete bipartite graph containing $2n$ vertices, where n vertices are contained in each partition?
8. Suppose you create a symmetrically normalized adjacency matrix $S = \Delta^{-1/2}A\Delta^{-1/2}$ for an undirected adjacency matrix A . You decide that some vertices are “important” and they should get relative weight $\gamma > 1$ in an embedding that is similar to that in spectral clustering, whereas other vertices only get a weight to 1.

- (a) Propose a weighted matrix factorization model that creates an embedding in which the “important” vertices have a relative weight of γ in the objective function. The matrix factorization model should yield the same embedding at $\gamma = 1$ as symmetric spectral clustering.
 - (b) Show how you can create a class-sensitive embedding with this approach, if some vertices in a graph are labeled.
 - (c) You are given a black-box classifier that works with multidimensional data. Show how you can select γ appropriately and use it for collective classification of the unlabeled vertices of a partially labeled graph.
9. Propose an embedding-based algorithm for outlier detection in multidimensional data that uses the concept of the similarity graph proposed in Section 10.8.2. Discuss the choice of an appropriate dimensionality of the embedding, and how this choice is different from the case of the clustering problem.
 10. Suppose you are given a very large graph for which the symmetric similarity matrix S (for spectral clustering) cannot be materialized in the disk space available to you. Discuss why the data-dependent nature of this kernel matrix makes Nyström sampling difficult. Propose a rough approximation of Nyström sampling for spectral clustering. [The answer to the last part of the question is not unique.]
 11. Provide an example of a 2×2 adjacency matrix of a directed graph that is not diagonalizable.
 12. A bipartite graph is defined as a graph $G = (V_1 \cup V_2)$ with a partitioned vertex set $V_1 \cup V_2$, so that no edges in E exist within vertices of a single partition. In other words, for all $(i, j) \in E$, both i, j cannot be drawn from V_1 , and both i, j cannot be drawn from V_2 . Show that if λ is the eigenvalue of the adjacency matrix of an undirected bipartite graph, then $-\lambda$ is an eigenvalue as well.
 13. **Degree conditioning:** Suppose that you have the adjacency matrix A of an undirected graph. Discuss using the Perron-Frobenius theorem why adding the (weighted) degree of each node i to the i th diagonal entry results in a positive semidefinite matrix. The text of the chapter already shows that negating each entry of A and adding the degree matrix results in a positive semidefinite Laplacian. Also provide an alternate proof of the positive semidefiniteness of the unnormalized Laplacian using the Perron-Frobenius theorem.
 14. Suppose that you are given the symmetric factorization $A \approx UU^T$ of an undirected (and possibly weighted) graph adjacency matrix in which all entries of A are well approximated by UU^T (with the possible exception of the zero diagonal entries). Furthermore, all entries of the $n \times k$ matrix U are nonnegative. Discuss how you can use this factorization to express A as the sum of k adjacency matrices, each of which is a rank-1 matrix. Discuss how you can use this decomposition to create a possibly overlapping clustering of the graph by inspection of U . Discuss the interpretability advantages of nonnegative matrix factorization.
 15. Let P be the stochastic transition matrix of an undirected and connected graph. Show that all left eigenvectors of P other than the principal left eigenvector (i.e., *PageRank* vector) have vector components that sum to 0. [Hint: What are the angles between left eigenvectors and right eigenvectors of a matrix?]

16. Let S be the symmetrically normalized adjacency matrix of spectral clustering of an undirected graph. In some cases, the clusters do not clearly separate out by applying the k -means algorithm on the features obtained from eigenvector extraction on S . Use the kernel intuition from Chapter 9 to discuss the advantages of using $(C+S) \odot (C+S)$ instead of S for eigenvector extraction in such cases. Here, $C \geq 0$ is a matrix of constant values. [Hint: The k -means algorithm works best with linearly separable clusters. Check Figure 9.4.]
17. Consider two $n \times n$ symmetric matrices A and B , such that B is also positive definite. Show that BA need not be symmetric, but it is diagonalizable with real eigenvalues. [Hint: This is a generalization of the proof that stochastic transition matrices have real eigenvalues by setting B to the inverse degree matrix and A to the adjacency matrix.]
18. Suppose that A is the 20×20 binary adjacency matrix of a directed graph of 20 nodes. Interpret the matrix $(I - A^{20})(I - A)^{-1}$ in terms of walks in the graph. Will this matrix have any special properties for a strongly connected graph? Argue algebraically why the following is true:

$$(I - A^{20})(I - A)^{-1} = (I - A)^{-1}(I - A^{20})$$

19. Exercise 13 of the previous chapter introduces symmetric non-negative matrix factorization, which can also be used to factorize the symmetrically normalized adjacency matrix $S \approx UU^T$, which is used in spectral clustering. Here, U is an $n \times k$ non-negative factor matrix. Discuss why the top- r components of each column of U (in magnitude) directly provide clustered bags of nodes of size r in the graph.
20. Find the *PageRank* of each node in (i) an undirected cycle of n nodes, and (ii) a single central node connected with an undirected edge to each of $(n - 1)$ nodes. In each case, compute the *PageRank* at a restart probability of 0.
21. **Signed network embedding:** Suppose that you have a graph with both positive and negative weights on edges. Propose modifications of the algorithms used to remove “weak edges” and to symmetrically normalize the graph for spectral clustering. Will the resulting graph be diagonalizable with orthogonal eigenvectors and real eigenvalues? Is there anything special about the first eigenvector? [This is an open-ended question with multiple solutions.]
22. **Heterogeneous network embedding:** Consider a social network graph with directed/undirected edges of multiple types (e.g., undirected friendship links, directed messaging links, and directed “like” links). Propose a shared matrix factorization algorithm (cf. Chapter 8) to extract an embedding of each node. How would you tune the parameters? [This is an open-ended question with multiple solutions.]